Causal Generality in the Sparse-Autoencoder Features of a Vision-Language-Action Model

A complete technical record of the experimental programme: motivation, methods, mathematics, results, and claims.

Project: MrVLA. Model under study: OpenVLA-7B, fine-tuned on the LIBERO benchmark. Reference work under examination: Swann, McGranahan, Buurmeijer, Kennedy & Schwager, *Sparse Autoencoders Reveal Interpretable and Steerable Features in VLA Models*.

Who this document is for

This is written for a reader who has done roughly one or two years of an undergraduate degree in a quantitative subject. That means we assume you are comfortable with:

- basic algebra and summation notation,
- the idea of a vector as a list of numbers,
- the idea of an average, and roughly what a correlation is.

We assume **nothing** about machine learning, robotics, transformers, sparse autoencoders, or advanced statistics. Every one of those is built up from scratch. Where a formula appears, it is written out, every symbol is named, and the reason we compute that particular quantity rather than some other one is explained.

The aim is that you finish this document able to say, for every number in the results, three things: what was measured, how it was measured, and why that measurement answers the question it claims to answer.

> **A note on difficulty.** We have not simplified the science. The experiments involve some genuinely intricate reasoning — particularly about *controls*, which are the parts of an experiment designed to rule out boring explanations for exciting-looking results. Where something is hard, we slow down and break it into pieces rather than skipping it. If a section feels dense, the payoff is usually in the paragraph that starts "**Why we do this**".

How this document is organised

Chapter	What it covers	Read it if you want to know
1	The machines and the vocabulary	What a VLA is, what an SAE is, what a "feature" means
2	The scientific question	What "general" means and why the existing definition fails
3	The mathematical toolkit	Every statistic and algorithm used, explained from zero
4	Phase 1 — building the measures	How the two measurement paths were built and first tested
5	Phase 2 — auditing the measures	Whether those numbers survive their own controls
6	Complete results	Every number, in tables
7	Claims	What we assert, and what we deliberately do not
8	Limitations and known issues	Everything currently wrong or unresolved
9	Glossary and appendices	Definitions, file map, reproduction instructions

Chapters 1–3 are background and can be skipped by a reader who already works in interpretability. Chapters 4–5 are the experimental record. Chapters 6–8 are the scientific content proper.

A one-paragraph summary, for orientation

A robot policy is a neural network that turns camera images and a written instruction into motor commands. Inside it, information is stored in a way that does not line up neatly with human concepts, so researchers use a tool called a *sparse autoencoder* to break that information into interpretable pieces called *features*. A natural question is which features are **general** — reusable across many situations — and which are **memorised** — tied to one specific situation. The existing way of answering that question turns out to measure something else entirely. We replace it with a definition based on *causal influence*: a feature is general to the degree that removing it would change the robot's chosen action across many different tasks. We show this definition is measurable, replicates across four task suites, and reveals that causal influence is concentrated in roughly 100 of 2048 features. We then show that this within-model notion of generality does **not** predict whether a differently fine-tuned copy of the same base model will contain the same feature — and that this failure is not an artefact of how we matched features, which was the most plausible innocent explanation.

Chapter 1 — The machines and the vocabulary

1.1 What problem is the robot solving?

Imagine a robot arm above a table. On the table are objects — a bowl, a plate, a cabinet with a drawer. The robot has a camera looking at the scene, and it is given an instruction in plain English, for example *"put the bowl on the plate"*.

Roughly twenty times a second, the robot must decide what to do next. Its decision is not "pick up the bowl" — that is far too abstract for a motor. Its decision is seven numbers:

```
dx how far to move the gripper left/right (a small distance)
dy how far to move it forward/backward (a small distance)
dz how far to move it up/down (a small distance)
droll how far to rotate about one axis (a small angle)
dpitch how far to rotate about a second axis (a small angle)
dyaw how far to rotate about a third axis (a small angle)
grip whether the gripper should be open or closed
```

Those seven numbers are called an **action**. The prefix *d* means "delta", i.e. a small change from where the arm is now, not an absolute position. Together the first six describe how to nudge the hand in three-dimensional space, and the seventh controls the fingers. This is called a **7-DoF action** ("degrees of freedom").

A **policy** is any function that takes the current situation and returns an action. Formally, if `o` is what the robot observes and `a` is the action:

```
a = pi(o) "pi" is the traditional symbol for a policy
```

Run the policy repeatedly — observe, act, observe, act — and you get an **episode**: a whole attempt at the task, which ends either in success (the bowl is on the plate) or failure (a time limit is reached, or something is knocked over).

1.2 What is a Vision-Language-Action model?

A **Vision-Language-Action model** (VLA) is a policy built out of a large language model. The idea sounds strange at first — language models predict text, they do not move robot arms — but the trick is simple: *turn the action into text*.

Concretely, in the model we study (**OpenVLA-7B**):

1. The camera image is passed through a vision encoder, producing a sequence of vectors that stand in for "what the scene looks like".
2. The instruction is turned into word-pieces (**tokens**) in the usual way for a language model.

3. Both are fed into a large transformer language model — in this case one built on Llama-2 with about 7 **billion** adjustable numbers (7B **parameters**).
4. The model then *writes out* the action, one token at a time, as if it were writing seven words.

The critical design choice is step 4. Each of the seven action numbers is **discretised**: the continuous range of possible values is chopped into **256 equal buckets**, called **bins**. Instead of outputting the number 0.037, the model outputs "bin 148". And because a language model can only emit tokens from its vocabulary, each of the 256 bins is permanently assigned to one token of the tokenizer's vocabulary — specifically the 256 least-used ones, which the model would otherwise almost never emit.

So producing one action means producing **seven tokens**, one per degree of freedom, generated one after another (**autoregressively** — each token is produced with the previous ones already visible to the model).

> **Why this matters for everything that follows.** The same 256 tokens are reused for all seven slots. There is no separate vocabulary for "up/down" versus "gripper". Which physical quantity a token refers to is determined **only by its position** in the sequence of seven. Token 148 in the first slot means a particular left/right movement; the identical token in the seventh slot means a particular gripper state. We will return to this repeatedly: *channel identity is positional*.

One more detail that will matter later. The mapping from bin to token id runs **backwards**:

```
token_id = vocab_size - bin_index bin_index runs 1, 2, ..., 256
```

so bin 1 gets the *highest* token id and bin 256 the lowest. Any analysis that treats the bin axis as ordered — for example "did this feature push the action up or down?" — has to undo that reversal, and if it does not, the error is silent.

1.3 LIBERO: the tasks

LIBERO is a standard benchmark of simulated tabletop manipulation tasks. It is divided into four **suites**, each containing **10 tasks**:

Suite	What varies between its tasks	Example flavour
`spatial`	The *layout*: same objects, different positions	"pick up the black bowl between the plate and the ramekin"
`object`	The *objects*: same layout, different things to manipulate	"pick up the alphabet soup and place it in the basket"
`goal`	The *goal*: same scene, different requested outcome	"open the middle drawer of the cabinet"
`10` (also called LIBERO-Long)	Everything — ten long, multi-step tasks	"put both the cream cheese box and the butter in the basket"

The authors of OpenVLA released four separately fine-tuned checkpoints, one per suite. **All four start from the same base OpenVLA model** and are then trained further on their own suite. This shared ancestry is important and we will come back to it: these four models are not independent, they are siblings.

A **rollout** is one run of the policy on one task from one starting configuration, producing an episode that succeeds or fails. Rollouts are how we test whether an intervention on the model changes its behaviour.

1.4 Inside the model: the residual stream

To interpret a model you have to look inside it. Here is the minimum you need.

A transformer processes a sequence of tokens through a stack of **layers** (OpenVLA has 32, numbered 0 to 31). At every layer, and for every position in the sequence, the model holds a vector of numbers. In our model that vector has **d = 4096** components. This running vector is called the **residual stream**, and we write it `h`.

The name comes from how layers work: each layer does not *replace* `h`, it *adds* to it.

```
h_after_layer = h_before_layer + (what this layer computed)
```

So the residual stream is like a shared blackboard that every layer writes onto. By the time you reach the final layer, `h` contains everything the model has worked out and is about to use to choose its next token.

Every experiment in this project reads `h` at layer 31 — the last layer — at the exact positions where the seven action tokens are being produced. That specific choice is the single most important methodological decision in the project, and §4.3 explains why an earlier choice had to be abandoned.

1.5 How the model turns `h` into a chosen action bin

This is the step that makes the whole causal analysis possible, so we do it slowly.

At the end of the network, the residual vector `h` is converted into a **score for every token in the vocabulary**. Those scores are called **logits**. The token with the highest logit is the one emitted (this is called taking the **argmax** — "the argument that maximises").

Two operations happen. First a normalisation called **RMSNorm** ("root mean square normalisation"):

```
r = sqrt( mean(h_i^2) + eps ) a single number: the "size" of h
RMSNorm(h) = (h / r) * g divide by that size, then scale
```

Here `eps` is a tiny constant (about 0.00001) that stops division by zero, and `g` is a fixed vector of 4096 learned numbers called the **gain**, applied component by component. In words: *shrink or stretch `h` so its typical component size is 1, then rescale each component by a learned amount*. The purpose is numerical stability during training.

Second, the **unembedding**. There is a big matrix `W_U` with one row per vocabulary token. The logit for token `t` is the **dot product** of the normalised residual with row `t`:

```
logit(t) = RMSNorm(h) . u_t where u_t is row t of W_U
```

If you have not met the dot product, it is defined in §3.1; for now, read `x . y` as "a single number measuring how much `x` and `y` point in the same direction".

We only ever care about the 256 action-token rows, which we collect into a matrix `W_U_act` of shape [256 x 4096].

> **The key structural fact.** The last operation the model performs is a **dot product**, and a dot product is **additive**: `(a + b) . u = a.u + b.u`. So if we can write `h` as a sum of pieces, we can write the logit as a sum of contributions, one per piece — and each contribution is a number we can compute exactly. This is why layer 31 is special. At any earlier layer, `h` still has to pass through many nonlinear operations before it becomes a logit, and no exact decomposition exists.

1.6 The problem interpretability is trying to solve

You might hope that the 4096 components of `h` each mean something — component 17 means "there is a bowl", component 892 means "the gripper is closed". They do not. Networks routinely store far more distinct concepts than they have components, by giving each concept its own **direction** in the 4096-dimensional space rather than its own axis. Directions can be packed in much more densely than axes, at the cost of slight interference between them. This phenomenon is called **superposition**.

So the concepts are there, but they are smeared across all 4096 numbers. Interpretability needs a way to un-smear them.

1.7 Sparse autoencoders

A **sparse autoencoder** (SAE) is the standard tool for this. The idea:

1. **Learn a dictionary.** Find a large set of directions in the 4096-dimensional space — call them `w_1, w_2, ..., w_F` — such that any real residual vector `h` from the model can be written as a sum of just a **few** of them.
2. **Encode.** Given `h`, compute a number `z_j` for each dictionary direction saying how strongly that direction is present. The vector `z = (z_1, ..., z_F)` is called the **code**, and its entries are called **feature activations**.
3. **Decode.** Rebuild an approximation of `h` by adding the directions back up, weighted by the code.

The autoencoder part means it is trained to reproduce its own input (`h` in, approximately `h` out). The **sparse** part is the whole point: we force most of the `z_j` to be exactly zero, so that only a handful of directions are used to explain any given `h`. That is what makes the pieces interpretable — a description in terms of 100 active items is legible, one in terms of 4096 is not.

In this project:

- `F = 2048` dictionary directions ("features"). Note this is *fewer* than the 4096 dimensions of `h` — an unusual choice inherited from the reference paper's recipe, so this dictionary is compressive rather than expansive.
- Sparsity is enforced by **TopK with k = 100**: compute a score for all 2048 features, keep the 100 largest, set the other 1948 to exactly zero. Simple and gives exact control over sparsity.
- The dictionary directions are stored as rows of a matrix `W_dec` of shape [2048 x 4096], each row scaled to have length 1 (**unit norm**).

The exact reconstruction the SAE performs is:

```
h ~= l2 * ( sum over j of z_j * w_j ) + mu * 1 + b_pre
```

with these pieces:

Symbol	Shape	Meaning
`z_j`	number	how strongly feature `j` is active on this input (zero for all but 100 features)
`w_j`	4096 numbers	the direction feature `j` writes into the residual stream (row `j` of `W_dec`)
`l2`	number	a per-input scale factor; this SAE normalises each input before encoding, and this undoes it
`mu`	number	the mean of the input's components, added back to every component (`1` means a vector of all ones)
`b_pre`	4096 numbers	a fixed learned offset, the same for every input

The terms `mu * 1 + b_pre` together form a **constant** part of the reconstruction that does not depend on which features fired. Remember it — it will turn out to matter enormously in Chapter 5.

1.8 What we mean by "a feature"

Throughout this document, **feature** `j` means: *the pair (direction `w_j`, activation `z_j`)*. Talking about "feature 1167" means index 1167 in this particular trained dictionary.

Three warnings about that, all of which the project has had to take seriously:

1. **Features are not neurons.** They are learned directions, not components of `h`.
2. **Feature indices are dictionary-specific.** Feature 1167 in the `goal` model's SAE has no relationship to feature 1167 in the `spatial` model's SAE. Comparing across models requires a matching procedure, and the design of that procedure is a whole research question (Chapter 4, §4.9 and §5.11).

3. **Feature identity is not fully reproducible.** Train the same SAE again with a different random starting point and you get a *different* dictionary that explains the same data roughly as well. Quantifying that instability is one of the project's findings (§4.10).

Chapter 2 — The scientific question

2.1 The intuition: general versus memorised

Suppose we look at feature 1167 in the `goal` model and find that it switches on whenever the robot is about to close its gripper — no matter which object, which layout, which task. That looks like a **general** feature: a reusable piece of machinery, a "grasp" primitive.

Now suppose feature 1140 switches on only during one specific task, when the arm is above one specific drawer in one specific scene. That looks **memorised**: a lookup-table entry for one situation.

The distinction matters for a practical reason. A policy built mostly out of general primitives should transfer to new situations. A policy built mostly out of memorised entries should be brittle — it will do well on what it was trained on and fail as soon as anything changes. Being able to *measure* which of the two a policy is doing would tell us something real about robot learning.

So: how do you measure it?

2.2 The existing answer, and why it does not work

The reference paper defines a general feature as one that **fires across many episodes for a human-nameable event**. In practice this is implemented as a classifier: compute a handful of statistics about *when* each feature fires, hand-label 30 features as general or memorised, fit a classifier from statistics to labels, and report its accuracy.

The statistics are things like:

- **coverage** — the fraction of episodes in which the feature fires at all;
- **mean onsets** — how many separate times it switches on within an episode;
- **mean magnitude** — how strongly it fires;
- **relative run length** — how long it stays on.

The reported accuracy is 100% under leave-one-out cross-validation. That sounds decisive. It is not, for two separate reasons.

Problem 1: the labels and the features are not independent

To be a valid test, a classifier's **labels** must be obtained independently of its **inputs**. Here they are not. The labelling procedure screens candidates using burstiness (one of the metrics), requires the global metrics to agree before a label is assigned, and excludes ambiguous cases. So the labels are partly *defined by* the same metrics the classifier is given as input.

This is a known failure mode with a name: **criterion contamination**. Combined with **range restriction** (throwing out the ambiguous middle makes any classification easier), 100% accuracy is close to guaranteed by construction. It measures label-metric consistency, not whether the construct "general" is real.

> This is not an accusation of misconduct. The paper documents its own failures honestly — two features it discusses at length are mis-classified by its own classifier. The point is a methodological one: the reported accuracy cannot be evidence for the construct.

Problem 2: the statistics reduce to "how often does it fire?"

This one we established ourselves, and it is the more damaging of the two.

We built two label-free firing-based metrics of our own and validated them properly, using **leave-one-group-out**: score a feature using 9 of the 10 tasks, then check whether that score predicts how the feature behaves on the held-out 10th task. Raw results looked encouraging — correlations of 0.3 to 0.8.

Then we applied the **confound control**. A confound is a third variable that could explain the result without the interesting mechanism being true. The obvious candidate here is a feature's overall **base firing rate**: how often it fires at all, across everything. A feature that fires a lot will *mechanically* appear in more episodes, more tasks, and more groups.

We therefore asked: does the score still predict held-out behaviour *once base firing rate is held fixed*? (The technique for holding a variable fixed is **partial correlation**, explained in §3.4.)

The answer was no, twice over:

- Under LIBERO's balanced task groups, one of the metrics is **algebraically identical** to base firing rate. Nothing remains after controlling for it. And by the same argument, **so is the paper's** `coverage`.
- The other metric, after controlling for base rate, correlated 0.94 with *concentration* and predicted held-out firing **negatively** in all 20 layer-by-suite cells tested (–0.05 to –0.27). Controlled for activity, it is a **memorisation** signal, not a generality one.

> **The lesson that reshaped the project.** A validation target of *firing* can never isolate generality, because firing **is** activity. Two equally busy features — one a genuine reusable primitive, one meaningless noise — receive the same score. The entire firing-statistics route is closed.

2.3 Our redefinition

If we cannot use firing, and we do not want human labels, what is left? **Causal influence.**

> **General** = a feature whose **causal influence on the policy's chosen action recurs across many tasks**. **Memorised** = a feature whose influence is confined to one task or situation. Generality is a **continuous spectrum**, not a binary label.

Two things this deliberately does *not* require:

- **No human interpretability.** A feature may be general without any person being able to name what it responds to. Whether general features also happen to be interpretable becomes an empirical question we report, not an assumption we build in.
- **No labels at all.** Everything is computed from the model's own behaviour.

Splitting one word into two axes

The old definition welded together two properties that are actually independent. Making them separate is the core conceptual move.

Axis	Question it asks	Name we use
Task-breadth (causal)	Does this feature *drive the action* across many different tasks?	**Path A**
Cross-model recurrence	Does this feature's causal role *reappear* in a separately fine-tuned model?	**Path B**

The example that forced the split is a "lid detector" discussed in the reference paper. It fires on every kind of lid, in every scene — so by any invariance-flavoured definition it is highly general. But there are only a couple of lid tasks, so its influence on the policy's behaviour is narrow. **High invariance, low breadth.** One number was averaging two orthogonal things.

A grasp detector scores high on both. A memorised feature scores low on both. A diffuse control signal with no clean concept scores high on breadth and low on invariance. Four distinct quadrants, previously collapsed into

one score.

2.4 The three methodological commitments

Fixed in advance, and the reason the project kept finding things:

1. **Falsifiable at every level.** A metric that fails a validity check is itself a result, publishable as such. This is why "the firing route is closed" is written up rather than buried.
2. **Confound-first.** No score is trusted until it is shown to predict something *beyond* base firing rate. This principle already killed one whole family of metrics (§2.2) and, as Chapter 5 shows, it caught a second confound years later in a completely different measurement.
3. **Agreement over reliability.** Precision is not validity. A number can be extremely repeatable and still measure the wrong thing. Confidence comes from *independent* measurements agreeing — different layers, different seeds, different models, different behaviours.

2.5 What a "control" is, and why this document is full of them

Most of the technical machinery in Chapters 3–5 exists to serve one purpose, so it is worth stating plainly.

Suppose you measure something and get an exciting number — say a correlation of +0.49. Before believing it means what you hope, you must rule out the boring explanations:

- **Is it just arithmetic?** Would you get +0.49 from *any* data of this shape, regardless of what the model does? → answered by a **null distribution** (§3.5): scramble the data in a way that destroys the effect but preserves everything else, recompute, and see what you get.
- **Is it just a known confound?** Would base firing rate alone produce it? → answered by a **partial correlation** (§3.4).
- **Is it just noise?** With this much data, how big a number would random chance produce? → answered by **confidence intervals** and **statistical power** (§3.6–3.8).
- **Could the experiment have detected the effect if it were there?** A null result from an experiment too small to see anything is not evidence of absence. → answered by the **minimum detectable effect** (§3.8).
- **Is the measurement reliable enough to support the claim?** Two very noisy measurements cannot correlate strongly even if the underlying truth is a perfect relationship. → answered by **reliability** and **attenuation** (§3.9).

Every one of those five appears in the results. Chapter 3 builds each tool; Chapters 4 and 5 apply them.

Chapter 3 — The mathematical toolkit

Every tool used anywhere in the project, built from the ground up. If you already know a section, skip it; the experiments in Chapters 4–5 refer back by number.

3.1 Vectors, dot products, length, and cosine

A **vector** is just an ordered list of numbers. The residual stream `h` is a vector with 4096 entries. We write `h_i` for the `i`-th entry.

The **dot product** of two vectors of the same length multiplies them entry by entry and adds up the results:

```
x . y = x_1*y_1 + x_2*y_2 + ... + x_n*y_n = sum over i of x_i * y_i
```

The **length** (or **norm**) of a vector is the square root of its dot product with itself — the multi-dimensional Pythagoras theorem:

```
||x|| = sqrt( x . x ) = sqrt( x_1^2 + x_2^2 + ... + x_n^2 )
```

A vector with length 1 is called a **unit vector**. **Normalising** a vector means dividing it by its own length, which keeps its direction and sets its length to 1.

The dot product has a geometric meaning: if `theta` is the angle between two vectors,

```
x . y = ||x|| * ||y|| * cos(theta)
```

Rearranging gives the single most-used quantity in this project, **cosine similarity**:

```
cos(x, y) = ( x . y ) / ( ||x|| * ||y|| )
```

Read it as **"how aligned are these two directions, ignoring how long they are?"**

Value	Meaning
`+1`	identical direction
`0`	perpendicular — completely unrelated directions
`-1`	exactly opposite directions

Two facts we lean on. First, if both vectors are already unit length, cosine similarity is *just the dot product*, so computing all pairwise cosines between two sets of vectors is one matrix multiplication. Second, in a high-dimensional space two *random* directions are almost always nearly perpendicular — the typical cosine between random vectors in `n` dimensions is about `1/sqrt(n)`, which for n = 256 is about 0.06. **So a cosine of 0.3 between two 256-dimensional vectors is not "weak"; it is far above what chance produces.** This is why every cosine in this report is quoted against a measured chance floor rather than judged on intuition.

3.2 Means, centring, and why we subtract things

The **mean** of a list is its average. **Centring** a list means subtracting its mean so the new list averages to zero.

Centring appears twice in this project and both times for the same reason: **to remove a component that carries no information about a choice.**

Suppose the model must choose between 256 action bins by picking the largest logit. Now suppose something adds the *same amount* to all 256 logits. The ranking is unchanged, so the choice is unchanged. That common component is irrelevant to the decision and should not be credited to anything.

So whenever we ask "how does this feature affect the choice?", we first subtract the average effect across all 256 bins. The resulting direction is called the **contrast direction**:

```
u_contrast(t) = u_t - (average of u_s over all 256 action tokens s)
```

A feature that lifts every action bin equally has zero contrast effect and correctly scores zero.

3.3 Correlation, ranks, and Spearman

Correlation measures whether two lists of numbers move together. The standard version (**Pearson**) is the cosine similarity of the two lists after centring:

```
r = sum_i (x_i - xbar)(y_i - ybar)
/ sqrt( sum_i (x_i - xbar)^2 * sum_i (y_i - ybar)^2 )
```

It runs from –1 (perfectly opposed) through 0 (unrelated) to +1 (perfectly aligned).

Pearson correlation assumes the relationship is a straight line and is easily distorted by a few extreme values. Our data has both problems — feature magnitudes are heavy-tailed, with a handful of features enormously larger than the rest. So we mostly use **Spearman correlation**, which is Pearson correlation applied to **ranks** instead of raw values.

Ranks: replace the smallest value by 1, the next by 2, and so on. A list of `(3.1, 900.0, 5.2)` becomes `(1, 3, 2)`. This throws away *how much* bigger something is and keeps only the ordering, which makes the measure immune to outliers and to any relationship that is increasing but curved.

> **A subtlety that bit us.** When two values are exactly equal ("tied"), textbook Spearman gives them the **average** of the ranks they would occupy. A common shortcut — sorting twice — instead breaks ties by whichever came first in the array, which injects an arbitrary ordering. That distinction is invisible on continuous data and very visible on counts. It is a live issue in this project; see §8.3.

3.4 Partial correlation: holding a variable fixed

This is the workhorse of the confound-first principle, so it gets a full explanation.

The question. We find that breadth correlates with importance. But busy features might score high on both simply *because* they are busy. We want: **does breadth still predict importance among features that fire equally often?**

Running the analysis only on features with identical base rates would throw away nearly all the data. The standard alternative is **residualisation**.

The recipe, for a control variable `c`:

1. Fit the best straight line predicting `x` from `c`. The **residual** `x_res` is what is left over: `x_res = x − (line's prediction from c)`. By construction `x_res` contains no linear information about `c`.
2. Do the same for `y`, giving `y_res`.
3. Correlate `x_res` with `y_res`.

The result is the **partial correlation of x and y controlling for c**. Read it as: *the relationship between x and y that is not explainable by c*.

We use the rank version throughout: rank all three variables first, then residualise. With two controls (`c1` and `c2`) the straight line becomes a plane, fitted by least squares, and the interpretation extends unchanged. In our code that is `rank_partial_both(y, x, c1, c2)`, and the two controls are always **causal magnitude** and **base firing rate**.

> **Why exactly those two controls?** Magnitude, because "this feature matters a lot in total" is a boring reason to look general. Base rate, because "this feature fires constantly" is the confound that destroyed the previous metric family. If a breadth score survives both, it is telling us something about *how influence is distributed across tasks* rather than about how strong or how frequent the feature is.

3.5 The participation ratio: turning a shape into a count

Suppose a feature has some amount of causal influence on each of 10 tasks — a list of 10 non-negative numbers. We want one number summarising **how many tasks it really spreads over**. Counting non-zeros is too crude: a feature with 9.99 units on task 1 and 0.001 on the others would count as 10.

The **participation ratio** solves this:

```
PR(x) = ( sum_i x_i )^2 / ( sum_i x_i^2 )
```

Work through the two extremes:

- All the mass on one task, `x = (5, 0, 0, ..., 0)`: numerator `25`, denominator `25`, so **PR = 1**.
- Mass spread evenly over 10 tasks, `x = (5, 5, ..., 5)`: numerator `50^2 = 2500`, denominator `10 * 25 = 250`, so **PR = 10**.

So PR runs from 1 (everything in one place) to `n` (perfectly even), and reads directly as **an effective count**.

Why the formula works. Normalise to proportions `p_i = x_i / sum(x)`. Then `PR = 1 / sum(p_i^2)`. The quantity `sum(p_i^2)` is the probability that two independent random draws land on the *same* item. If mass is concentrated, collisions are likely and PR is small; if spread out, collisions are rare and PR is large. PR is the reciprocal of a collision probability, which is exactly what "effective number of items" should mean.

PR is scale-free: multiplying every entry by 100 leaves PR unchanged. That is essential — it measures *breadth*, not *strength*, which is what lets us separate the two.

We use PR in four different roles, and it is worth keeping them straight:

Applied over	Gives	Where
tasks, per feature	effective number of tasks a feature drives = **breadth**	§4.7
features, per model	effective number of features carrying the action = **concentration**	§5.3
action channels, per feature	effective number of the 7 action dimensions a feature drives	§5.9
tasks, per ablation condition	effective number of tasks a removal damages	§5.5
singular values	**effective rank** of a matrix	§3.13

3.6 Null distributions and permutation tests

The question a null answers. You measured +0.49. Would you have got +0.49 from data with no real structure?

The method. Take your real data and scramble it in a way that destroys the effect you claim while leaving everything else intact. Recompute the statistic. Do this a thousand times. You now have a **null distribution**: the range of values the statistic takes when the claimed effect is absent. Compare your real number to it.

If your value lands far outside, the effect is real. The **p-value** is simply the fraction of scrambled runs that reached your value or beyond.

Everything depends on choosing the right scramble. A permutation that destroys too much makes any result look significant; one that destroys too little makes a real result look like nothing. The project made *both* mistakes at different times and caught both:

- An early cross-model null permuted the 256 action-bin axes. That destroyed not only feature correspondence but the *shared geometry of the readout itself*, deflating the floor and manufacturing a fake gap of about 0.33 (§4.11).
- The plan's prescribed control for Path A permuted task labels within a feature. That destroyed **nothing the statistic depends on**, and reproduced the real value almost exactly (§5.2).

> **The rule of thumb.** A good null destroys exactly the correspondence you are claiming, and preserves every other structural property — marginal distributions, matrix shapes, the number of things being maximised over.

3.7 Confidence intervals, and the Wilson interval

If you observe 152 successes out of 200, the success rate is 0.76 — but the *true* rate could be somewhat different by luck. A **95% confidence interval** is a range constructed so that, over many repetitions of the experiment, the true value falls inside it 95% of the time.

The textbook interval `p ± 1.96 * sqrt(p(1-p)/n)` fails badly near 0 or 1. At 20 successes out of 20 it gives `1.00 ± 0.00` — claiming you know the rate perfectly from 20 trials, which is absurd.

The **Wilson score interval** fixes this by solving for which true rates are consistent with the observation, rather than assuming a symmetric spread around the estimate:

```
centre = p + z^2/(2n)
margin = z * sqrt( p(1-p)/n + z^2/(4n^2) )
interval = ( centre - margin , centre + margin ) / ( 1 + z^2/n )
```

with `z = 1.96` for 95%. At 20/20 it gives roughly [0.84, 1.00] — asymmetric, and honest.

3.8 Paired data, McNemar's test, and statistical power

Why pairing matters

Suppose you test whether removing a feature hurts the robot. You run 200 episodes normally and 200 with the feature removed, and compare success rates. But episodes differ enormously in difficulty — some starting positions are nearly impossible. Much of the difference between your two numbers is *which starting positions each condition happened to get*.

Unless you make them the same. Our ablation runs **replay the identical starting configurations** under every condition. Episode 7 of task 3 has the same initial state in the baseline and in every ablated condition. That makes the data **paired**, and pairing removes the episode-difficulty variation entirely — a large gain in sensitivity for free.

McNemar's test

With paired binary outcomes, every pair falls into one of four boxes:

	ablated succeeded	**ablated failed**
baseline succeeded	both fine	`b10` — **damage**
baseline failed	`b01` — **repair**	both failed

The two diagonal boxes are **concordant**: both conditions did the same thing, so they say nothing about which is better. Only the **discordant** counts `b10` and `b01` carry information.

Under the null hypothesis that the ablation has no effect, each discordant pair is equally likely to fall either way — a coin flip. So with `m = b10 + b01` discordant pairs, `b10` should follow a **binomial distribution** with `m` trials and probability 0.5. **McNemar's test** is just asking how unusual the observed split is under that coin-flip model. For small `m` we compute this exactly; the common chi-squared approximation is over-conservative below about 25 discordant pairs and would throw away real power.

The **damage** and its confidence interval:

```
d = ( b10 - b01 ) / n
SE = sqrt( (b01 + b10) - (b10 - b01)^2 / n ) / n
95% interval = d +- 1.96 * SE
```

Note that concordant pairs contribute nothing to the uncertainty either — the whole test lives on the discordant pairs.

Statistical power and the minimum detectable effect

This is the concept that turned one of our uninterpretable results into a reportable one, so it deserves care.

A null result — "we saw no effect" — has two completely different possible causes:

1. There is no effect.
2. There is an effect, but the experiment was too small to see it.

You cannot tell these apart from the result alone. What distinguishes them is **power**: the probability that the experiment would detect an effect of a given size if that effect were real. Conventionally we ask for 80% power.

Turning this around gives the **minimum detectable effect** (MDE): the smallest effect the design would catch 80% of the time. Anything smaller than the MDE is *below the resolution of the experiment*, and observing nothing there says nothing at all.

For a paired binary test, the derivation runs like this. Conditional on `m` discordant pairs, we are testing whether a coin is fair. The test rejects when

```
|p - 0.5| * sqrt(m) > z_alpha * 0.5 + z_power * sqrt( p(1-p) )
```

where `p` is the true probability a discordant pair goes the damage way, `z_alpha = 1.96` and `z_power = 0.84`. Writing the damage as `d = (2p − 1) * disc_rate`, where `disc_rate = m/n` is the fraction of pairs that are discordant, and taking the worst case `p ≈ 0.5` in the variance term gives the clean form:

```
MDE ~= ( z_alpha + z_power ) * sqrt( disc_rate / n )
~= 2.80 * sqrt( disc_rate / n )
```

Worked example from our data. With `n = 200` pairs and a measured discordance rate of 0.253:

```
MDE = 2.80 * sqrt(0.253 / 200) = 2.80 * 0.0356 = 0.0996 -> about 10 percentage points
```

So a 200-episode design detects a 10-point drop and nothing smaller. Rearranging for the sample size needed to detect a target effect `d`:

```
n = disc_rate * ( ( z_alpha + z_power ) / d )^2
```

For a 5-point effect at the same discordance rate: `n = 0.253 * (2.80/0.05)^2 = 793` pairs, i.e. **79 episodes per task** across 10 tasks instead of 20.

3.9 Reliability, split-half, Spearman–Brown, and attenuation

Reliability

Reliability is the extent to which a measurement agrees with *itself* when repeated. It runs from 0 (pure noise) to 1 (perfectly repeatable). It is not the same as validity: a broken thermometer that always reads 20°C is perfectly reliable and completely invalid.

Split-half

To measure the reliability of a score computed from 10 tasks, split the tasks into two disjoint halves, compute the score **independently on each half**, and correlate the two versions across features. Repeat over many random splits.

Two details that matter. **Everything derived from the tasks must be recomputed per half** — if you reuse a quantity computed on all 10 tasks, the held-out half leaks in and the reliability is inflated. And a **floor** is needed: shuffling feature identity in one half should give ≈0, confirming the procedure is calibrated.

Spearman–Brown

A half-length measurement is *less* reliable than the full one, so the raw split-half correlation understates the truth. The **Spearman–Brown** formula corrects for length:

```
r_full = 2 * rho / ( 1 + rho )
```

where `rho` is the observed half-to-half correlation. Example: `rho = 0.222` gives `r_full = 0.444/1.222 = 0.363`. The formula is meaningless for `rho <= 0`, which we treat as "no reliable signal".

Attenuation: the ceiling on any correlation

Here is the fact that makes reliability essential rather than decorative. **Noisy measurements cannot correlate strongly, even when the underlying truth is a perfect relationship.** Formally, if `x` and `y` have reliabilities `r_xx` and `r_yy`, then the correlation you observe relates to the true one by

```
r_observed = r_true * sqrt( r_xx * r_yy )
```

Since `r_true` can be at most 1, this gives a hard **ceiling**:

```
| r_observed | <= sqrt( r_xx * r_yy )
```

Two consequences, and **they point in opposite directions**, which is the part that is easy to get wrong:

- **For a positive result**, the correction is free and helps you. Attenuation only ever *shrinks* a correlation, so dividing by the ceiling gives a **lower bound** on the truth. If you measured +0.49 with a breadth reliability of 0.363, then `|r_true| >= 0.49/sqrt(0.363) = 0.82`. The true relationship is *at least* that strong.
- **For a null result**, the correction does **not** help you, and assuming otherwise is a real error. To claim a null you need an **upper** bound on `|r_true|`, and the formula gives you a lower one. Substituting `r_yy = 1` for an unknown reliability produces `|r_true| >= |r_obs|/sqrt(r_xx)` — an argument *against* the null. Defending a null requires knowing that the **ceiling was high**: the measurement had room to show a big correlation and returned a small one.

When one reliability is unknown, the honest move is to invert the question and report the **breakeven reliability** — how reliable the other measure would have to be for the observed value to bound the truth below some threshold:

```
require |r_true| = |r_obs| / sqrt(r_xx * r_yy) <= threshold
rearrange r_yy >= r_obs^2 / ( r_xx * threshold^2 )
```

3.10 The bootstrap

To attach uncertainty to a statistic with no neat formula, **resample the data with replacement** many times, recompute the statistic on each resample, and read off the middle 95% of the results.

Which unit you resample defines what uncertainty you are describing. If you resample tasks, you are asking "what if I had drawn a different set of tasks?" If you resample episodes, you are asking "what if the same tasks had gone differently by luck?"

If both sources of variation are real, you need a **two-level bootstrap**: resample tasks, and then within each drawn task resample its episodes, recomputing the per-task quantity from the resampled episodes. Omitting the second level treats a noisy per-task number as if it were known exactly, and produces intervals that are far too confident.

> A counterintuitive consequence, verified on synthetic data: propagating extra noise does **not** always widen the interval. Extra noise in `x` pulls `corr(x, y)` toward zero, which concentrates the bootstrap replicates near zero. The two-level interval can be *narrower* while being centred nearer zero. **Coverage of zero, not width, is the property to check.**

3.11 Concentration: Lorenz shares and the Gini coefficient

To describe how unequally a total is split among many items, two standard measures:

Top-N share — the fraction of the total held by the N largest items. "The top 50 of 2048 features carry 45% of the causal mass."

Gini coefficient — a single summary of inequality from 0 (everything perfectly equal) to 1 (one item has everything). Sorting the values ascending as `v_1 <= ... <= v_n`:

```
Gini = 2 * ( sum_i i * v_i ) / ( n * sum_i v_i ) - (n + 1)/n
```

We report Gini, top-N shares and PR-based effective counts together because each is misleading alone.

3.12 Singular values and effective rank

A matrix maps vectors to vectors. The **singular value decomposition** (SVD) finds a set of perpendicular directions such that the matrix acts on each by simple stretching. The stretch factors are the **singular values** `s_1 >= s_2 >= ...`.

If a matrix has 256 rows but its singular values collapse to near zero after the tenth, then although the rows live in a 4096-dimensional space they really only span about ten directions. That "really only spans about" number is the **effective rank**, and we measure it with the participation ratio applied to the *energies* (squared singular values):

```
effective_rank = ( sum_i s_i^2 )^2 / ( sum_i s_i^4 )
```

We also report the **rank at 90% and 99% of energy**: the smallest number of directions needed to account for that share. The two disagree when the spectrum is "heavy-headed" — a couple of dominant directions drag the participation ratio down while many small directions still carry real structure. When they disagree, neither should be trusted alone.

> **Why we care.** Cross-model matching compares directions in a shared space. If that space is effectively 8-dimensional, then *any* handful of directions spans nearly all of it, "matching" becomes trivially easy, and a positive result would be dimension-counting rather than a finding. Measuring the effective rank before running the experiment tells you whether the experiment can mean anything.

3.13 Matching algorithms

Three related problems appear in Path B and its extensions.

Greedy best-match. For each item in set A, take its single best partner in B. Simple, and what the original recurrence metric does. Its weakness: several A-items can grab the *same* popular B-item, which inflates apparent agreement.

Hungarian assignment. Find the one-to-one pairing of A-items to B-items that maximises the *total* similarity, with no reuse. This is a classical optimisation problem with an exact polynomial-time solution (the shortest-augmenting-path algorithm). It is stricter than greedy, and the gap between the two is itself informative: a big gap means a few popular partners were absorbing many matches.

Orthogonal matching pursuit (OMP). Sometimes the right question is not "which single B-item matches this A-item?" but "can a *combination* of B-items reproduce it?" OMP answers this greedily:

1. Start with the target direction as the current residual.
2. Pick the B-item most correlated with the residual.
3. Project the target onto the space spanned by everything picked so far; subtract to get a new residual.
4. Repeat `m` times.

After `m` steps you have the cosine between the target and its best approximation from `m` chosen items. Sweeping `m` from 1 upward asks *how much a coalition helps*. Two properties to keep in mind:

- **Greedy is not optimal.** A locally best first pick can lead away from the best group of `m`. So the curve is a **lower bound** on what a coalition could achieve. We mitigate this with restarts (force the first pick to be each of the top few candidates and keep the best result).
- **Cosine rises with `m` automatically.** More directions span more of anything. So a rising curve proves nothing unless compared against a null run at **the same** `m` and with the same dictionary size.

3.14 Clustering directions on a sphere

k-means partitions points into `k` groups, each represented by its centre, by alternating: assign every point to its nearest centre; recompute each centre as the mean of its members; repeat.

Our points are *directions* — a feature's causal signature — where length means strength, not identity. So we normalise every point to unit length and use cosine similarity instead of straight-line distance; centres are re-normalised each round. This is **spherical k-means**. Empty clusters are re-seeded rather than dropped, so that `k` really means `k` and cluster counts remain comparable between models.

3.15 Comparing distributions without clustering: sliced Wasserstein

Clustering forces a choice of `k`, and a conclusion that changes with `k` is a conclusion about `k`. As a cross-check we compare two clouds of directions with no clustering at all.

The **Wasserstein distance** (or "earth mover's distance") between two distributions is the minimum work needed to reshape one into the other. In one dimension it has a simple form: sort both samples and compare them at matched quantiles. In many dimensions it is expensive — so the **sliced** version projects both clouds onto many random one-dimensional directions, computes the easy 1-D distance on each, and averages:

```
SW(A, B) = average over random directions p of W_1( A projected on p , B projected on p )
```

Smaller means the two clouds are distributed more alike. Like every other similarity in this report, it is only interpretable against a floor and a ceiling.

Chapter 4 — Phase 1: building and first-testing the measures

This chapter is the experimental record of the original programme: how the two measurement paths were built, what had to be rebuilt when a measurement turned out to be invalid, and what the first round of results said.

4.1 The order things happened in, and why

The programme did not proceed in a straight line, and the detours are part of the record.

Stage	What it was	Outcome
Firing metrics	Two label-free metrics from firing statistics	**Null** — they are base firing rate (§2.2)
Viability check	Does the existing data support causal analysis?	**No** — wrong vectors were collected (§4.2)
A1	Re-collect the right vectors	Done
A2	Retrain the SAE on them	Done
A3	Gate: is causal decomposition even valid here?	**Pivoted**, then passed (§4.5)
A4	Measure causal breadth, control confounds	**Positive** (§4.8)
Path B	Cross-model recurrence	Positive but weak, and reframed (§4.10–4.12)
A×B	Do the two axes agree?	**They do not** (§4.13)
Behaviour	Ablation and steering	Ablation **null**, steering works (§4.14)

4.2 The data problem that forced a rebuild

The original activations were collected with a hook that did two things which turned out to be fatal for causal analysis:

1. it **mean-pooled** the residual vectors across all the prompt tokens, averaging them into one vector per input; and
2. it captured only the **prefill** pass — the model reading the prompt — not the passes where action tokens are actually produced.

Both are reasonable for asking *"what does the model represent about this input?"*. Both are useless for asking *"what causes this action?"*, because the vectors involved are literally not the ones the model uses to choose the action. An average over prompt positions never enters the action readout at all.

> **Why this is worth recording rather than quietly fixing.** The entire causal programme is only meaningful because it operates on the exact vectors that feed the readout. Discovering that the existing dataset did not contain them meant an expensive re-collection and an SAE retrain before a single causal number could be computed. It also means that the earlier Path B results, computed on the pooled data, describe a genuinely different object from the later action-position ones.

4.3 A1 — collecting the right vectors

The re-collection captures, for every decision the policy makes:

Quantity	Shape	Meaning
`residual`	7 x 4096	layer-31 residual at each of the 7 action-token positions, un-pooled
`token_ids`	7	which action token was actually emitted at each position
`task_id`, `episode`, `timestep`	1 each	bookkeeping, enough to locate any decision

and once per model, the constants the readout needs: the 256 action rows of the unembedding `W_U_act`, the final-norm gain `g`, and `eps`.

One easily-missed detail was handled explicitly. OpenVLA's action tokens are **not** simply the last 256 rows of the unembedding matrix. The matrix is padded to a round size, and the model decodes actions using `token_id = vocab_size − bin_index` where `vocab_size` is the *unpadded* size. Selecting the last 256 rows would have picked the wrong tokens and made every attribution silently wrong.

The final dataset for the `goal` suite: **446,096 decisions** (63,728 timesteps x 7 action slots), across 10 tasks.

4.4 A2 — retraining the SAE

The SAE must be trained on the *same distribution* it will be used to analyse, so a new dictionary was trained on the action-position residuals: `F = 2048` features, TopK with `k = 100`, 250 epochs, at layer 31.

4.5 A3 — the viability gate, and the pivot

Before measuring anything, we must check that a linear decomposition of the action is *valid at all*. If the SAE's features cannot account for the model's action choice, then attributing the action to features is meaningless and the project should stop and report that.

The first gate, and why it failed

The original gate (**L1**) was: take the SAE's reconstruction of the residual, push it through the real RMSNorm and unembedding, and ask whether it produces the **same argmax** over the 256 action tokens as the true residual. Threshold: 85% agreement.

It stalled at **0.72**. Training 150 more epochs moved it from 0.70 to 0.72. Raising sparsity from `k = 100` to `k = 256` reached only 0.76, on a slope so shallow that clearing 0.85 would have needed `k` in the many hundreds — abandoning sparsity, and with it the entire point of the SAE.

Diagnosis. L1 asks the SAE to reconstruct the *entire* 4096-dimensional residual well enough to win an argmax. But that residual carries information for the whole 32,000-token vocabulary, almost all of which is irrelevant to the action. The SAE was being penalised for failing at a task nobody needed it to do, and argmax over 256 tightly-packed bins is a brittle target.

The pivot: sufficiency

The right question is narrower. We do not care whether the SAE reproduces the residual. We care whether **the features account for the part of the readout that decides the action**.

Recall from §3.2 that only the *contrast* matters — the component that distinguishes bins. Define the **action margin** of a decision as the contrast-projected logit of the token actually emitted:

```
true = RMSNorm(h) . u_contrast(emitted token)
```

Now substitute the SAE's decomposition of `h`. Because the readout is a dot product, the margin splits **exactly** into three additive pieces:

```
true = features + bias + error

features = sum over j of (l2/r) * z_j * <w_j , g (*) u_contrast>
bias = ( mu*1 + b_pre ) . ( g (*) u_contrast ) / r
error = whatever the SAE failed to reconstruct
```

where `(*)` is entry-by-entry multiplication. **Sufficiency** asks what fraction of `true` each piece supplies, measured as a straight line through the origin fitted by least squares:

```
S(component) = sum over decisions of ( true * component )
/ sum over decisions of ( true * true )
```

This is "the fraction of the margin this component recovers". A value of 1.0 means the component accounts for the whole margin; 0.5 means half. The three components' slopes sum to exactly 1 by construction, because they sum to `true` itself.

> **Why a slope and not a correlation.** A correlation would say whether the component *tracks* the margin, ignoring size. We need to know whether it *supplies* the margin. A component that is perfectly correlated with the true margin but ten times too small has correlation 1.0 and slope 0.1, and only the slope tells you the truth. The through-origin form also avoids dividing by small numbers, which a per-decision ratio would do.

Result

On the `goal` suite at `k = 100`, with no retraining:

Component	Slope	Reading
features + bias	**0.9361**	93.6% of the action margin is additively recovered — the gate passes at 0.80
features alone	0.531	just over half the margin comes from which features fired
constant bias	0.405	**a large fixed component**, independent of the input
error	0.064	what the SAE missed
arithmetic canary	1.0000	mean absolute discrepancy 2.7e-14

The last row is a **bug canary**, not a scientific result: it checks that the frozen-`r` arithmetic reproduces the SAE's own reconstruction exactly. At 2.7e-14 — floating-point dust — the implementation is verified.

Two things to carry forward. First, the gate passed, so attribution is meaningful. Second, and unexpectedly, **about 40% of the action margin is a constant that does not depend on the input at all** — a fixed lean

toward a default action. At the time this was noted as a curiosity. Chapter 5 identifies exactly what it is.

> **A commitment honoured.** Changing a metric after seeing a bad result is a classic way to fool yourself. The record states explicitly that L1 is still reported beside sufficiency, that the metric was changed for a stated reason (L1 measures the wrong thing), and that the change was made before reading the sufficiency value.

4.6 A4 — the attribution formula, derived

Now we can compute, for one decision, how much each individual feature contributed to the action.

Start from the contrast-projected margin and substitute the SAE decomposition:

```
RMSNorm(h) . u_c = ( (h/r) (*) g ) . u_c
= (1/r) * ( h (*) g ) . u_c
```

and with `h = l2 * sum_j z_j w_j + mu*1 + b_pre`, the dot product distributes over the sum:

```
= sum over j of (l2/r) * z_j * ( (w_j (*) g) . u_c ) + constant term
```

Each term in that sum is one feature's contribution. So:

```
phi_j = (l2 / r) * z_j * < w_j , g (*) u_contrast >
```

Piece	What it is	Why it is there
`z_j`	how strongly feature `j` fired	a feature that did not fire contributes nothing
`<w_j, g (*) u_contrast>`	**alignment**: how much this feature's direction points at the winning bin rather than the average bin	a feature can fire hard and still contribute nothing if it points sideways
`l2 / r`	per-decision scale factors	the SAE normalises each input; `r` is the RMSNorm size. Dropping either makes every `phi` wrong by a per-decision factor

> **Why this escapes the activity confound by construction.** The old firing metrics could not distinguish a busy feature from an important one. Here a feature can have a huge `z_j` and still have `phi_j ≈ 0`, because its direction is orthogonal to the contrast. **Firing and influencing are now separate quantities.** They are not independent — `phi` contains `z` — so the confound controls in §4.8 remain necessary, but the metric is no longer *definitionally* activity.

A computational note that also guarantees correctness: the alignment term `<w_j, g (*) u_contrast(t)>` does not depend on the decision at all, only on which token `t` was emitted. So it can be computed once for all 2048 features and all 256 tokens, giving a matrix we call the **causal signature** `S`, and every decision just looks up a column. The same matrix is central to Path B (§4.12), which is not a coincidence — it *is* the object that describes what a feature does to actions.

4.7 Breadth: from per-decision influence to a per-feature score

Aggregate `phi` into a **per-task causal importance**:

```
C_j(g) = average over all decisions in task g of | phi_j |
```

The absolute value is deliberate: we are asking how much the feature *matters*, not in which direction it pushes. This yields a matrix `C` with one row per task and one column per feature — for `goal`, 10 x 2048.

Breadth is then the participation ratio (§3.5) over the tasks:

```
PR_j = ( sum over tasks g of C_j(g) )^2 / ( sum over tasks g of C_j(g)^2 )
```

1 = all influence in one task. 10 = influence spread evenly over all ten.

Adjusted breadth

Raw PR is entangled with the confounds — a feature that fires everywhere has nonzero `|phi|` everywhere and mechanically scores high. Measured: `PR` correlates **+0.82** with base firing rate. So raw PR is not the claim.

Adjusted breadth is PR rank-residualised on both **total causal magnitude** and **base firing rate** (§3.4). It answers: *does this feature spread its influence over more tasks than its strength and its firing rate alone would predict?* This adjusted score is what all downstream feature selection uses.

4.8 The A4 result: leave-one-task-out with confound control

A correlation computed on all 10 tasks at once could be driven by a single unusual task, and it would be measuring an in-sample fit rather than a prediction. So the test is **leave-one-task-out** (LOTO):

For each task `g` in turn:

1. Recompute breadth **and** magnitude using only the other 9 tasks.
2. Take the held-out task's causal importance `C(g)` as the target.
3. Compute the rank-partial correlation of training-breadth with held-out-importance, controlling for training-magnitude and base rate.

Recomputing magnitude per fold matters: reusing the all-10-task magnitude would leak the held-out task into the control variable.

Result on `goal` (446,096 decisions, 10 tasks, 2048 features):

Quantity	Value	
Mean PR	6.05 tasks (10th percentile 2.65, 90th 9.91)	
`partial	both controls`	**+0.493**
Folds positive	**10 of 10**	
Worst fold	+0.399	
Standard deviation across folds	0.053	

Reading. Breadth of causal influence predicts a feature's importance on a task it was never measured on, and it does so beyond both confounds, in every fold. It is a real, out-of-sample, firing-independent property.

> **What this result explicitly is not.** It is a claim about an *axis*, not about individual features. At this stage no feature had been identified, and no *count* of general features existed — a count needs a principled cut-point, which was never established. The attribution is also first-order (`r` is held fixed), so it describes the direct readout effect, not the full nonlinear behaviour of the policy.

4.9 Making it concrete: identifying features and looking at frames

An axis is abstract. To check it picks out something real, we ranked features by adjusted breadth, took the extremes, found the decisions where each had the largest `|phi|`, and pulled the actual camera frames from those moments.

- **High adjusted breadth**: grasp events, release/open events, return-to-home pose. The same feature fires at "grasp" across many different scenes and objects — reusable manipulation primitives.
- **Low breadth, strong**: single-scene, single-phase detectors — one particular lid grasp in one particular layout. The memorisation end.

A distinction worth keeping: these general features are general *in when they fire* (across scenes) but specific *in what they do* (one primitive). That distinction will matter in Chapter 5.

A bug was caught here by eye. The "specialist" end was initially selecting features that were weak *and* narrow — features with no causal impact at all, which is a different thing from narrow-but-decisive. Selection now draws both ends from features that are causally eligible in the first place. The frames revealed it; a regression test now pins it.

4.10 Path B — does a feature's role reappear in another model?

Path A measures breadth *within* one model. Path B asks a different question: fine-tune the same base model on a different task suite, and does the same causal role show up again?

The procedure:

1. **Shared probe.** Push the *same* set of frames through all four fine-tuned models and record the residuals.
2. **Encode** each model's residuals with *that model's own* SAE, giving an activation matrix per model.
3. **Match.** For feature `i` in model A, find `q_i = max over j of corr(activations of A's feature i, activations of B's feature j)`. A high `q_i` means feature `i` has a twin in B.

Three controls were mandatory, and all three passed:

Control	What it rules out	Result
Permutation null — shuffle probe-frame order, preserving each feature's marginals *and* the best-of-2048 maximum	that matching is trivially easy	gap `q_cross − q_perm` = **+0.13 to +0.26 in all 20 (layer x suite) cells**
Base rate + inheritance — regress the recurrence ranking on firing rate and on what the shared base checkpoint contributes	that it is just activity, or just shared ancestry	only **0.1%–11.3%** of the ranking is explained by both together
Discrimination — does it separate features, or score them all alike?	that "everything recurs" trivially	spread from 0.27 (10th percentile) to 0.69 (90th)

The last one also answers the obvious objection that all four models share a base checkpoint: if mere network similarity drove the result, every feature would recur equally. They do not.

Calibration. Raw match quality has no natural scale, so it is expressed as **chance-corrected retention** against a ceiling:

```
ret_cc = ( q_cross - q_perm ) / ( q_seed - q_perm )
```

where `q_seed` is how well the *same* model's dictionary matches a **re-trained copy of itself with a different random seed**. That is the most any cross-model comparison could achieve. Measured: **0.587 / 0.619 / 0.635 / 0.585 / 0.473** at layers 0 / 8 / 16 / 24 / 31. So changing the entire fine-tuning suite costs only about 40% relative to merely changing the random seed. Recurrence peaks in the middle of the network and is lowest at the output layer.

4.11 An unplanned finding: SAE dictionaries are only ~60% seed-reproducible

Computing the ceiling produced a result nobody was looking for. Matching the `goal` model's SAE against a second SAE trained on **exactly the same activations** with only a different random seed gives:

```
q_seed = 0.640 / 0.657 / 0.629 / 0.612 / 0.531 at layers 0 / 8 / 16 / 24 / 31
```

Not ~1.0. **A model re-analysed with the same data does not perfectly re-find its own features.** This is a standalone methods result: any conclusion drawn from a single SAE fit inherits that instability, and reported feature indices must be read in that light.

It also has a second consequence, exploited in §4.13: a measurement that is only 60% reproducible cannot correlate strongly with anything.

Second unplanned finding, from the same machinery: the correlation between recurrence and the reference paper's `P(general)` score, computed on identical features, lies between **−0.17 and +0.17** across all 20 cells. Given §2.2's finding that the paper's score *is* base firing rate, this says a firing-based generality score tells you essentially nothing about whether another model rediscovers a feature.

4.12 Redefining recurrence properly — and a null that was wrong

The activation-matching metric above has a defect that emerged on inspection: recurrence plotted against breadth came out **U-shaped**, high at *both* the specialist and general ends, even after controlling for base rate and inheritance.

Diagnosis. Correlation-matching over activations rewards features with sharp, low-entropy firing patterns, because those are easy to match. That is a property of a feature's *firing shape*, not of its causal role. The U-shape is a **distinctiveness artefact**.

The fix: match in a common output space

Instead of matching *when* features fire, match *what they do to actions*. Using the causal signature from §4.6:

```
S[j, t] = < w_j , g (*) u_t > contrast-centred across the 256 bins
```

This is a 256-number description of how feature `j` pushes every possible action bin. Crucially, **the 256 bins mean the same thing in every model** — bin `t` is the same commanded action everywhere — so signatures are comparable across models even though feature indices are not. Match by cosine similarity.

The null that was wrong, and why

The first null permuted the 256 bin axes. That seemed natural and was badly wrong.

Every model's signatures are produced by pushing directions through an action head, and all four heads are nearly identical. **All signatures therefore live in essentially the same low-dimensional region of the 256-bin space.** Permuting bins rotates one model *out* of that shared region, destroying geometry that is genuinely shared rather than coincidental. The floor collapses and a large fake gap appears — empirically about **0.33 even between unrelated random models**.

The correct null destroys only feature *correspondence* while preserving the shared readout geometry: draw **random decoder directions**, push them through the same head, and match those. If real features match no better than random directions pushed through the same head, the gap is correctly zero.

Result

- **Chance floor = 0.226.** Everything sits above it: shared causal structure across models is real.
- The U-shape is **gone**, confirming it was the activation-matching artefact.
- Recurrence now **declines monotonically with breadth**. Specialists recur slightly *more*; the most general features recur *less*.
- Top-decile (most general) versus the rest: **goal −0.017, spatial −0.015, object −0.008, libero-10 −0.011** — all small, all negative.

4.13 The A x B join

Putting the two axes together on the same features:

```
corr( adjusted breadth , causal-signature recurrence ) = -0.127
```

Uncorrelated, to mildly inverse. The same conclusion emerged from a per-group comparison and a dissociation test. Combined with §4.11's independent firing-based result, **"recurrence is not generality" was established three separate ways**: via the firing metric, via activation recurrence, and via causal-signature recurrence.

At this stage the finding carried an explicit hedge: since SAE dictionaries are only ~60% seed-reproducible, a weak correlation between two noisy measurements is hard to interpret. Chapter 5 turns that hedge into a number.

4.14 Behaviour: ablation and steering

Everything so far is measured *inside* the model. The behavioural test asks whether it predicts what the robot does.

Ablation removes a feature during a rollout, using a hook that projects the feature's direction out of the residual stream on every forward pass:

```
h_ablated = h - ( h . V^T ) V V = the unit decoder directions being removed
```

Five conditions were run on `goal`, 20 episodes per task, every condition replaying the **same** initial states:

Condition	Which 5 features	Success rate
baseline	none	0.760
general	highest adjusted breadth	0.710
specialist	lowest adjusted breadth, among load-bearing	0.785
random	5 arbitrary features	0.755
firing	top 5 by raw firing rate	**0.020**

The intended contrast — general versus specialist — produced nothing. The `firing` condition destroyed the policy, but it is not magnitude-matched to the others and plausibly removes a large share of the residual norm rather than action-specific structure.

A pre-registered follow-up ablated six named features **individually**, each carrying a specific prediction about *which task* should break. Every one of them came back null.

Steering adds a multiple of a feature's direction instead of removing it. Steering toward a grasp-associated feature made the gripper close **earlier** in the episode — a clear behavioural effect.

> **The state of play at the end of Phase 1.** Two internally-measured positives (Path A breadth, Path B above-chance recurrence), one clear dissociation between them, one methods finding about SAE instability — and a behavioural test that produced nothing interpretable. Whether that behavioural null meant "the features do not matter" or "the experiment was too small to tell" was, at this point, **unknown**. Answering that is where Chapter 5 begins.

Chapter 5 — Phase 2: auditing what those numbers mean

Phase 1 produced numbers. Phase 2 asks, of each one: **would this number look the same if the interesting thing were not true?** Every experiment in this chapter is a control, an extension designed to break a Phase 1 result, or a measurement of how much a Phase 1 result can bear.

Everything here is re-analysis of data already collected, except one GPU pass (§5.8) and one SAE retrain.

5.1 Why audit at all

Three specific worries motivated it.

1. **Path A's +0.493 had no floor.** It was positive in every fold, which is reassuring about consistency but says nothing about whether +0.493 is a large number *for a statistic of that shape*. Some statistics are positive on nearly any data.
2. **The ablation null was uninterpretable.** As stated at the end of Chapter 4, "we saw nothing" had two possible causes and no way to distinguish them.
3. **The A x B null rested on two noisy measurements.** §3.9 shows that unreliable measures *cannot* correlate strongly. Without knowing the reliabilities, `−0.127` might mean "these are different things" or "we cannot see anything from here".

5.2 Floors for Path A — and a control that does nothing

The two valid floors

The claim is that breadth predicts held-out importance. Two different boring explanations need ruling out, so two different scrambles.

Column shuffle — *the one that matters*. Independently permute **feature identity within each task row** of the matrix `C`. This preserves every task's marginal distribution of causal mass, and preserves the purely mechanical fact that a column with evenly-spread values has a predictable held-out entry. What it destroys is a feature having an *identity across tasks*. Whatever survives is arithmetic, not biology.

Feature shuffle — the estimator floor. Permute feature identity of the held-out vector only, leaving the predictor and both control variables untouched. Nothing links predictor to target, so anything other than ~0 would mean the estimator itself is biased.

Both nulls call the identical function that produces the reported number, so there is no possibility of scoring a re-implementation that has drifted from the original.

Result — all four suites

Suite	`partial | both`	Folds +	Worst fold	Column-shuffle floor	z	Feature-shuffle floor	z
goal	**+0.493**	10/10	+0.399	+0.0004 (sd 0.0094)	+52.1	+0.0002	+71.4
spatial	**+0.449**	10/10	+0.333	−0.0000 (sd 0.0094)	+47.9	+0.0004	+66.2
object	**+0.387**	10/10	+0.359	+0.0000 (sd 0.0096)	+40.2	−0.0003	+57.1
libero-10	**+0.535**	10/10	+0.399	−0.0000 (sd 0.0096)	+55.8	+0.0001	+75.4

(`z` is how many null standard deviations the observed value sits above the floor; $p < 0.001$ in all cells with 1000 permutations.)

Reading. The mechanical floor is zero. Path A replicates in all four suites at 40 to 75 standard deviations above chance. The A5 replication table, previously a promise, is now a measurement.

The control the plan prescribed does nothing

The written plan called for permuting **task labels** and expected the statistic to collapse to ~0. It does not, and the reason is instructive.

LOTO already evaluates **every** fold. For feature `j`, holding out task `g` contributes the pair (breadth computed on the other 9, importance on `g`). Permuting column `j` by some permutation makes fold `g` contribute the pair that fold `perm(g)` contributed in the real data. Across all folds it is **the same set of pairs, dealt into different folds**. Since the fold-level results are all positive and similar in size, mixing them reproduces the statistic.

Measured on the real data:

Suite	Observed	Task-label permutation
goal	+0.4926	**+0.4840**
spatial	+0.4488	**+0.4508**
object	+0.3866	**+0.3843**
libero-10	+0.5347	**+0.5327**

Why this is worth reporting rather than quietly correcting. Had the control been run as written, it would have returned "the null equals the result" and looked like a catastrophic failure of the headline finding. It is a genuine methods trap: an intuitive permutation that destroys nothing the statistic depends on.

5.3 Concentration and reproducibility, quantified

The Phase 1 write-up rebutted "of course hundreds of features influence the action" with the claim that **concentration and reproducibility** of influence, not influence itself, is the point. Neither word had a number.

What we compute. From the per-feature total causal magnitude:

- `n_eff` — participation ratio over *features* (§3.5): the effective number of features carrying the action.
- Top-N shares and Gini (§3.11).
- **Cross-task overlap** — the average number of features shared between the top-50 sets of two different tasks, expressed as a ratio to chance. Two independent 50-subsets of 2048 features share `50^2/2048 = 1.22` by chance.

Two controls: the same statistics computed for a **base-firing-rate** ranking, and for a **column-shuffled** matrix.

Suite	Effective #features (of 2048)	Top-10 share	Top-50 share	Gini	Top-50 cross-task overlap
goal	**102.3** (5.0%)	0.276	0.448	0.684	41.4/50 = **33.9x chance**
spatial	**105.9** (5.2%)	0.256	0.436	0.644	44.8/50 = **36.7x**
object	**49.3** (2.4%)	0.355	0.494	0.724	43.3/50 = **35.5x**
libero-10	**9.6** (0.5%)	0.588	0.731	0.889	42.7/50 = **35.0x**

Controls, both passed in every suite:

- **Firing rate concentrates far less** — for `goal`, `n_eff` 886.8 and Gini 0.449, against causal 102.3 and 0.684. So concentration is a *causal* statement, not an activity one.
- **Column-shuffled causal mass** gives `n_eff` 653.6 for `goal`. So the concentration comes from **the same features being large across tasks**, not from the shape of the marginal distribution.

Per-task `n_eff` is small in every individual task (`goal`: minimum 81.9, median 93.9, maximum 114.8), so this is not an artefact of averaging ten differently-shaped tasks.

Flagged for review. libero-10's `n_eff` of 9.6 is extreme — ten features effectively carrying a ten-task long-horizon suite. The column-shuffle control (91.2) says the concentration is real rather than marginal-driven, but a number that strong should be examined directly before publication.

5.4 How reliable is breadth?

Split-half (§3.9): split the 10 tasks into disjoint halves, recompute PR **and** magnitude independently on each half, Spearman-correlate across features, 200 random splits, Spearman-Brown corrected. A feature-identity shuffle pins the floor.

Suite	Raw PR reliability	Adjusted-breadth reliability
goal	0.221	**0.363**
spatial	0.349	**0.273**
object	0.637	**0.469**
libero-10	0.424	**0.337**

The floor is ~0.000 everywhere, so the procedure is calibrated and these are real but low.

Reading, carefully. These two statements are both true and not in tension:

- The **population-level axis is solid**. That is what §5.2 shows: breadth predicts held-out importance far above any floor, in every suite. Aggregated over 2048 features, a weak per-feature signal still produces a decisive population result.
- The **feature-level ranking is only weakly reproducible**. Ask "is feature 1167 more general than feature 1140?" and the answer changes substantially depending on which half of the tasks you measured on.

This bounds everything that depends on picking *individual* features — which includes the ablation and steering target selection.

5.5 Was the ablation null real, or was the experiment too small?

Phase 1's ablation reported point estimates and nothing else. We recompute with paired tests, intervals, and a power bound (§3.8).

`goal` **coalition run** — baseline 152/200 = 0.760, Wilson interval [0.696, 0.814]:

Condition	Success	Damage	95% CI	b10/b01	p (exact)	MDE at 80% power
firing	0.020	**+0.740**	[+0.679, +0.801]	148/0	< 0.001	0.169
general	0.710	+0.050	[–0.019, +0.119]	30/20	0.203	0.097
random	0.755	+0.005	[–0.065, +0.075]	26/25	1.000	0.098
specialist	0.785	–0.025	[–0.088, +0.038]	18/23	0.533	0.087

The design's resolution, from a median discordance rate of 0.253:

```
pooled (200 pairs) MDE = 2.80 * sqrt(0.253/200) = 0.097 -> 9.7 points
per task (20 pairs) MDE = 2.80 * sqrt(0.253/20) = 0.226 -> 22.6 points
to detect 5 points n = 0.253 * (2.80/0.05)^2 = 793 pairs = 79 episodes/task
```

Reading. General's +5.0 points was **never detectable** by this design. The honest statement is:

> General-coalition damage is below 11.9 points (the upper end of its interval); anything under 9.7 points was outside this run's resolution. The experiment does not show the features are inert.

That is a **bounded null** — a reportable result — rather than an absence of evidence.

The single-feature run is in the same position: pooled MDE 10.8 points, per-task 22.1. **Every pre-registered named-task prediction was untestable at 20 episodes per task.** The closest to an effect was one feature at –6.4 points [–13.3, +0.5], p = 0.108 — and in the *repair* direction, i.e. removing it slightly helped.

The **scope test** (does general-coalition damage spread over more tasks than specialist damage?) landed inside its sampling-noise null for every condition: it did not resolve either. For the single-feature run, damage participation ratio was undefined, because no task showed positive damage at all.

> **Discrepancy noted.** An earlier working note records the coalition baseline at 78.9%. The data files give 152/200 = 76.0%. The measured value is used throughout; the discrepancy is unexplained and is likely a partial versus complete run.

5.6 Can the A x B null be defended?

Applying the attenuation correction (§3.9) with the measured breadth reliability of 0.363:

Quantity	Value	What it means		
Observed `corr(breadth, recurrence)`	–0.127	the raw finding		
Measurement ceiling `sqrt(r_xx * r_yy)`	at most ±0.602	the largest correlation these measures could ever show		
Implied `	r_true	` lower bound	≥ **0.211**	attenuation only shrinks, so the truth is at least this
Breakeven recurrence reliability	**0.494**	recurrence must be at least this reliable for `	r_true	≤ 0.30`

Reading: at the feature level, the null is not currently defensible. Recurrence reliability has never been measured. Until it is, `–0.127` is consistent with a real correlation of –0.21 or considerably larger, and the claim "recurrence is not generality" is a statement about noisy measurements rather than about models. (§5.12 addresses this at a level where the objection largely dissolves.)

The same correction applied to Path A works entirely in its favour:

```
|r_true| >= 0.493 / sqrt(0.363) = 0.819
```

5.7 How many distinguishable causal roles are there? (the geometry gate)

Everything in the next three sections compares directions in the 256-bin signature space. Before running any of it, we measured how big that space actually is — because if it is tiny, then any handful of directions spans nearly all of it, matching becomes trivial, and a positive result would be dimension-counting rather than a finding.

Method: SVD (§3.12) of the contrast-centred `W_U_act`, and of the signature matrix the model's features actually occupy.

Object	Effective rank	90% energy	99% energy
Contrast-centred readout `W_U_act`	205.5	213	250
Signature space occupied by goal's 2048 features	**50.8**	193	246

The second row is the governing number. A random 12-dimensional subspace explains only **5.8%** of an arbitrary direction in a 50-dimensional space, so a coalition sweep out to m = 12 is interpretable. **The geometry is healthy; the experiment can mean something.**

A correction the gate produced

The gate also checks an assumption that had been asserted rather than tested: that all four models share an unembedding matrix. **They do not.** `g` and the action token ids are identical, but `W_U_act` differs by up to 0.0266 — about 1.64 times the standard deviation of its own entries.

The assumption came from reading this project's LoRA fine-tuning script, which trains only attention projections. But the four models are the **OpenVLA team's published checkpoints**, not products of that script, and their unembeddings were modified.

What this does and does not break. Each model's decoder must be pushed through **its own** head. The comparison itself remains valid, because the shared object is the 256 action **bins** — bin `t` is the same commanded action in every model — not the linear map onto them. It is the map that varies, not the semantic space.

And the practical impact is small: pushing the same decoder through another model's head gives a signature cosine of **0.9967 to 0.9990** (worst single feature 0.964). Reported so that a raw weight difference is not mistaken for a meaningful one.

5.8 Measuring necessity exactly, without rollouts

A rollout gives one bit per episode and, as §5.5 showed, resolves nothing below ten points. But at layer 31 the readout is the *entire* remaining computation, so the counterfactual "what would the model have chosen without this feature?" can be computed **exactly, in closed form, with no forward pass at all**.

The derivation

Write the unnormalised logits `L_t = (h (*) g) . u_t`. Removing a feature changes `h` by some multiple of its direction, so

```
h' = h - coeff * w_j
L'_t = L_t - coeff * S[j, t] S = the causal signature matrix from 4.6
```

Three consequences make this cheap and exact:

1. **The RMSNorm factor `r` drops out of the argmax.** It is a positive number that scales every action logit equally, so it cannot change which bin wins. (It is still needed if you want logit *values*, but not for "did the choice change?".)
2. **Contrast-centring `S` is irrelevant here too** — it subtracts a constant from every bin of a row, shifting all logits equally.
3. `S` **is the same matrix Path B uses** for cross-model matching. One object serves both analyses.

So a **flip** — the feature's removal changing the emitted action — is exactly `argmax(L − coeff * S[j]) != argmax(L)`.

Two different ablations, which are not the same intervention

This is a distinction the project had not previously drawn, and it turns out to matter.

Name	What it removes	`coeff`	Matches
projection	the whole component of `h` along the feature's direction	`<h, w_j>`	what the rollout hook does
coded	only the amount the SAE attributed to that feature	`l2 * z_j`	what `phi` describes

Projection removes everything lying along that direction — including SAE reconstruction error, the constant term, and overlap leaking in from other features. Coded removes only the feature's own coded contribution. Phase 1's rollouts used projection; Phase 1's attribution described coded. Nobody had checked whether they

agree.

Validation

Every path is verified against brute-force recomputation through the real RMSNorm and unembedding. On the real data, one further check is decisive: in coded mode a feature that did not fire has `coeff = 0` and **cannot** flip anything. Measured, coded flips on non-firing decisions come to –**0.5% of the total**, i.e. exactly zero within rounding. That single number validates the flip counting, the activity mask, and the accumulator wiring simultaneously.

Scale: **396 candidate features x 446,096 decisions = 176.7 million exact counterfactuals.**

> **Scope, stated precisely.** This is the exact **direct** effect on one decode slot at a **frozen** state. It excludes two other routes by construction: (a) within a timestep, the gripper slot attends to the six action tokens already emitted, so a feature could influence it indirectly; and (b) across timesteps, changing the action changes which state the robot next occupies. Both are real and both are invisible here. §5.10 returns to this.

5.9 B1 — is generality located in one action channel?

The hypothesis

Phase 1's frame inspection said the general features look like grasp, release, and return-to-home detectors. All three are fundamentally **gripper and phase** events. That suggests a sharp, falsifiable prediction: causal breadth should concentrate in the gripper channel.

Testing it requires recovering an axis that Phase 1 discarded. The attribution code aggregates `|phi|` over tasks and, in doing so, sums over all 7 action slots. Recomputing while keeping the slot gives a per-channel causal matrix.

Two traps, handled explicitly

The scale trap. Raw `|phi|` is **not comparable across slots**. `phi` carries `u_contrast`, whose length depends on where the emitted bin sits in the ordered range. The gripper is near-binary and emits extreme bins, which have the largest contrast norm — so *every* feature looks stronger there for a purely geometric reason. The fix is to work in **shares**: a feature's fraction of the total `|phi|` at that decision and slot. Both forms are computed, and a conclusion appearing only in the absolute numbers is treated as the confound rather than the finding.

The degeneracy trap. The gripper token is constant for most of an episode. A feature can score high there simply by dominating a low-entropy channel. So every statistic is also computed restricted to **gripper-transition timesteps** — the moments the command actually changes.

Validation. The recomputed argmax matches the token the model actually emitted on **99.19%** of decisions. (The 0.81% mismatch is most plausibly float16 storage of the residuals flipping near-tie decisions; it does not corrupt the flip analysis, which uses the recomputed argmax as its own consistent baseline.)

Result: the hypothesis is falsified

`corr(adjusted breadth, gripper concentration)`, rank-residualised on magnitude and base rate: **+0.069**. The same partial for all seven channels:

dx	dy	dz	droll	dpitch	dyaw	gripper
+0.081	+0.083	**+0.195**	+0.089	+0.099	+0.162	**+0.069**

All small, all positive, and the gripper is the **lowest of the seven**. The general/specialist channel profiles differ by at most 0.07 with `P(general > specialist)` between 0.51 and 0.62. **There is no channel story.** Generality is not localised to a channel.

> That all seven partials come out positive is itself odd, since the seven concentrations sum to 1 per feature and ought to partly cancel. That pattern is more consistent with a shared artefact than with seven channel-specific

effects, and is listed as an open issue (§8.5).

A positive by-product: channel breadth is a second axis

The participation ratio over the 7 channels gives the **effective number of action dimensions** a feature drives: mean **3.21 of 7** (10th percentile 1.33, 90th 6.32). Its correlation with task breadth is only **+0.238**.

So there are two largely independent breadth axes — across tasks, and across action dimensions — where the project previously had one.

The gripper is a default, not a decision

Per-slot sufficiency (§4.5, computed per channel):

	dx	dy	dz	droll	dpitch	dyaw	gripper
features + bias	0.940	0.934	0.905	0.949	0.920	0.937	**0.992**
features alone	0.607	0.833	0.593	0.903	0.923	0.776	–**0.046**

Six channels are feature-driven. The gripper's action margin is recovered almost entirely by the **constant** `mu + b_pre` **bias**, and the features contribute nothing.

This identifies what §4.5's unexplained finding — "a constant default-action bias of 0.405" — actually *is*. It is largely the gripper.

Curiously, features put **more** `|phi|` mass on the gripper slot than on any other (0.154 of the run's total, the highest of the seven). They push hard on the gripper logits; their pushes simply do not align with which bin wins. **Loud, but not decisive.**

Which half of this evidence is safe. The sufficiency slope on a near-binary channel is partly tautological: both the true margin and the bias term are functions of the emitted token, so "the bias predicts the margin" comes close to being circular where only two tokens occur. The **flip rate is not tautological** and agrees: given the feature fires, removing it changes a `dx` token 0.0534 of the time and a gripper token **0.0008** — about 65 times less. On gripper-transition timesteps the gripper rate rises roughly ninefold (0.0008 to 0.0075) but remains far below translation. Lead with necessity; cite sufficiency as corroboration.

5.10 What a projection ablation actually does

Using both semantics from §5.8 over the full 176.7 million counterfactuals:

	Flip rate, all decisions	Flip rate, given the feature fires
projection	0.0229	0.0695
coded	0.0060	0.0642

Features are active on 9.4% of feature-decisions. Decomposing projection's flips:

```
total flips = 0.0229 * 176,654,016 = 4.05 million
when the feature fires = 0.0695 * 16,586,298 = 1.15 million
when it does NOT fire = 2.89 million = 71.5 %
```

71.5% of what a projection ablation does happens on decisions where the feature never fired. Given the feature *does* fire, the two interventions are nearly equivalent (0.0695 versus 0.0642) — the entire gap is the non-firing decisions.

This is a direct caveat on reading any projection-based rollout ablation as evidence about *coded* features, and Phase 1's rollouts were projection-based.

A confound caught in our own numbers

Over all decisions, coded flips showed general 0.0036 versus specialist 0.0007 — a 5.1x separation that looked like strong support for the breadth ranking. Conditioning on the feature actually firing:

Group	projection	coded
general	0.0534	0.0569
random	0.0565	0.0564
specialist	0.0292	0.0342
firing	0.1284	0.1150

The 5.1x collapses to 1.66x, and **random is indistinguishable from general**. Most of the apparent separation was base rate: a feature that fires twice as often has twice the opportunity to change an action, and in coded mode a feature that does not fire cannot flip anything at all. This is exactly the confound that destroyed the firing metrics in §2.2, reappearing in a new measurement.

> **Reading this carefully.** "General ≈ random" is not straightforwardly a failure of the breadth ranking. `adjusted_breadth` is *residualised on magnitude by construction* (§4.7), so it was never designed to predict per-decision strength — it claims *breadth* of influence, which is a scope claim, and per-decision flip rate is not a scope measurement. The informative half is that the **specialist** end is genuinely less decisive.

5.11 A4 — is the Path B null just an artefact of one-to-one matching?

The alternative explanation worth taking seriously

Path B matched features **one-to-one**: `q = max over j of cos(signature of A's feature i, signature of B's feature j)`. That is structurally blind to the best-documented failure mode of sparse dictionaries — **splitting**. §4.11 established that dictionaries are only ~60% seed-reproducible, and the usual reason is that one fit's feature becomes two or three in another.

Crucially, splitting is **not uniform across features**. Broad, diffuse, high-usage general features are exactly the ones expected to fragment; sharp specialists stay atomic. **So splitting alone could have produced "generals recur less" with no difference in recurrence underneath.** This is the strongest innocent explanation available, and it needed testing.

The test

Replace "does feature `i` have a twin?" with "**can model B's dictionary express feature `i`'s role using `m` features?**" — orthogonal matching pursuit (§3.13), sweeping `m` from 1 to 8. At `m = 1` this reproduces the published metric, so the experiment *extends* the existing result rather than replacing it.

The nulls must be `m`-matched. Cosine rises with `m` mechanically, so a null run at a different `m`, or against a dictionary of a different size, would manufacture a positive result out of dimension counting. Three controls all run at every `m`: random decoders through the corresponding model's own head; the same model under a different SAE seed (the ceiling); and, when available, the base model.

Two biases point in opposite directions and are both recorded. Unrestricted coefficients **overstate** what B can express, since TopK codes are non-negative and B can only *add* a feature's direction, never subtract it. Greedy matching pursuit **understates** it, since a locally-best first pick can lead away from the best group — on planted data where three features provably span the target, greedy recovers only 0.82 to 0.98 of it.

Result: splitting is not the explanation

	m=1	m=8	slope
most-specialist decile	0.3065	0.6052	+0.299
most-general decile	0.2828	0.5881	+0.305

	m=1	m=8	slope
random floor	0.2464	0.5684	**+0.322**
seed ceiling	0.4528	0.6974	+0.245

Every decile rises at essentially the same rate, and **the random floor rises faster than any of them**. Chance-corrected retention is flat across `m`: **0.244, 0.269, 0.272, 0.271, 0.268, 0.264, 0.259, 0.253**. Letting a model use eight features instead of one to express a role buys nothing.

Replicated with each suite as the target

Target	General slope	Specialist slope	Difference	Random-floor slope	General − floor
goal	+0.3053	+0.2987	+0.0066	+0.3220	**−0.0167**
spatial	+0.3040	+0.2947	+0.0093	+0.3213	**−0.0173**
object	+0.3049	+0.2985	+0.0064	+0.3218	**−0.0169**
libero-10	+0.3166	+0.3102	+0.0064	+0.3219	**−0.0053**

Two facts hold in all four suites, and the second is the stronger:

1. The general-versus-specialist slope difference is +0.006 to +0.009 — about 2–3% of the slope itself. Splitting does not preferentially help broad features anywhere.
2. **The random-floor slope is +0.322 in every suite and no decile reaches it.** Coalition matching does not merely fail to help real dictionaries; they gain *less* from it than random ones do.

One exception, recorded. The `m = 1` ordering reproduces the published direction in goal, spatial and object, but **libero-10 reverses it** — its most-specialist decile is the lowest. libero-10 also has the highest anti-aligned fraction against every partner (0.25 to 0.37) and the extreme concentration flagged in §5.3. It behaves differently on three separate axes and should be treated separately rather than averaged in.

5.12 A4 continued — comparing role inventories instead of features

If individual features do not match, perhaps the *set of roles* does. Cluster each model's signatures on the sphere (§3.14), match the cluster centres across models by Hungarian assignment (§3.13), and compare.

Three readings are produced. **Do the inventories match?** — centroid similarity against the same floor and ceiling. **Do the models weight roles the same?** — occupancy, the share of the dictionary each model devotes to a matched role, which is what duplication-style splitting would show up as. **Does the answer depend on `k`?** — cross-checked by a sliced Wasserstein distance (§3.15) that uses no clustering at all.

k	Inventory match	Random floor	Seed ceiling	Retention	`corr(role breadth, match quality)`
8	0.270	0.203	0.547	+0.194	**−0.810**
16	0.266	0.160	0.540	+0.279	**−0.574**
32	0.227	0.170	0.397	+0.251	**−0.537**

General-dominated roles match across models worse than specialist-dominated ones, at every `k`. And the two independent methods agree on magnitude: cross-model recurrence retains about **a quarter** of what an SAE seed change retains (m-sweep 0.24–0.27, clusters 0.19–0.28).

This also substantially blunts §5.6's reliability objection *for this claim*. That objection applied to a feature-level correlation measured at reliability 0.363; here each point aggregates 64 to 256 features, where measurement noise is far smaller.

An anomaly, unexplained

The sliced-Wasserstein column does something we cannot account for. **Random dictionaries sit closer to goal's signature distribution (0.0180–0.0182) than any real model does** (object 0.0186, spatial 0.0208, libero-10 0.0232; seed ceiling 0.0140). At the level of the whole distribution of directions, the four fine-tuned models differ from each other *more* than they differ from noise. This is recorded as an open problem (§8.5).

5.13 Defects found during Phase 2

Auditing turns up mistakes, including our own. The complete list:

Defect	Where it came from	Status
The prescribed A4 negative control is a no-op	the written plan	corrected (§5.2)
`--target` was a label-only argument, so a per-suite comparison table could be silently mislabelled	Phase 1 tooling	fixed
Rank ties broken by array index instead of averaged	Phase 1 tooling, inherited	**open** (§8.3)
The attribution-agreement bootstrap ignored episode-level noise	introduced in Phase 2	fixed (§5.14)
The shared-unembedding assumption	introduced in Phase 2	fixed (§5.7)
The claim that SAEs were trained on pooled residuals	introduced in Phase 2	**wrong, retracted** — they are un-pooled action-position residuals

5.14 One Phase 2 statistic that was wrong, and how it was caught

Worth documenting because it is a good illustration of §3.10.

The first version of the "does damage land where attribution said?" interval resampled **tasks** while treating each task's damage as a fixed number. At 20 episodes, a per-task damage carries a standard error near 0.16 — larger than most of the damages being correlated. The interval was therefore far too confident, and it produced `corr(damage, attribution) = −0.770` with a 95% interval of [–0.96, –0.45] for the general coalition: an apparently decisive finding that damage lands in the *opposite* place from where attribution predicted.

The fix is a two-level bootstrap that also resamples matched episode pairs within each drawn task. Tested on planted data where damage is **pure noise** but happens by chance to correlate strongly with the profile: across every seed where the task-only interval wrongly excluded zero, the two-level interval put zero back inside, while a genuine effect still excluded it. One such seed goes from [–0.95, –0.61] to [–0.87, +0.02] — almost exactly the pattern of the real result.

So the –0.770 was noise correlated with noise. Given the per-task MDE of 22.6 points and observed per-task damages of ±5 to 20 points, every per-task damage value in that run *is* noise, which is exactly what the corrected interval now says.

Chapter 6 — Complete results

Every number produced by the programme, in one place. Phase 1 figures are as reported in the project record; Phase 2 figures were computed or recomputed directly from the stored data during the audit.

6.1 Measurement-validity findings (what does not work)

Finding	Evidence	Status
The reference paper's classifier is circular	Labels are screened by the same metrics the classifier regresses onto; ambiguous cases excluded. 100% LOO accuracy measures label-metric consistency, not construct validity	Argument, not measurement
Group-balanced coverage **is** base firing rate	Algebraically identical under LIBERO's balanced task groups; partial correlation after controlling base rate is empty	Measured
The paper's `coverage` is base firing rate	Same argument applies	Argument
`max_group_rate` is a *memorisation* signal	After controlling base rate: correlates 0.94 with concentration; predicts held-out firing **negatively**, –0.05 to –0.27, in all 20 layer x suite cells	Measured
Pooled prefill activations cannot support causal analysis	They are not the vectors that feed the action readout	Structural

6.2 Path A — causal task-breadth

The viability gate (`goal`, k = 100)

Component	Slope	Note
features + bias	**0.9361**	gate threshold 0.80 — **PASS**
features alone	0.531	
constant bias	0.405	later identified as largely the gripper (§5.9)
error	0.064	
frozen-`r` arithmetic canary	1.0000	mean abs discrepancy 2.7e-14
abandoned L1 argmax gate	0.72	stalled; 0.76 even at k = 256

The core result, four suites

Suite	`partial | both`	Folds +	Worst fold	Column-shuffle floor	z	p
goal	**+0.493**	10/10	+0.399	+0.0004	+52.1	< 0.001
spatial	**+0.449**	10/10	+0.333	–0.0000	+47.9	< 0.001
object	**+0.387**	10/10	+0.359	+0.0000	+40.2	< 0.001
libero-10	**+0.535**	10/10	+0.399	–0.0000	+55.8	< 0.001

Supporting figures (`goal`): 446,096 decisions; PR mean 6.05 (p10 2.65, p90 9.91); raw PR correlates +0.82 with base firing rate, which is why the adjusted version is used; fold standard deviation 0.053; attenuation-corrected `|r_true| >= 0.819`.

Invalid control, for the record: task-label permutation gives +0.4840 / +0.4508 / +0.3843 / +0.5327 — indistinguishable from the observed values.

Concentration and cross-task reproducibility

Suite	`n_eff` (of 2048)	Top-10	Top-50	Top-100	Gini	Top-50 overlap	Per-task `n_eff` (min/med/max)
goal	102.3	0.276	0.448	0.529	0.684	41.4/50 = 33.9x	81.9 / 93.9 / 114.8
spatial	105.9	0.256	0.436	0.516	0.644	44.8/50 = 36.7x	91.6 / 102.4 / 109.1
object	49.3	0.355	0.494	0.570	0.724	43.3/50 = 35.5x	45.3 / 48.5 / 55.6
libero-10	9.6	0.588	0.731	0.780	0.889	42.7/50 = 35.0x	9.3 / 9.5 / 10.3

Controls (`goal`): firing-rate ranking gives `n_eff` 886.8, top-50 share 0.140, Gini 0.449. Column-shuffled causal mass gives `n_eff` 653.6, top-50 0.208, Gini 0.482. Chance overlap for top-50 of 2048 is 1.22.

Reliability of the breadth ranking

Suite	Raw PR (Spearman–Brown)	Adjusted breadth (Spearman–Brown)	Shuffle floor
goal	0.221	0.363	+0.002
spatial	0.349	0.273	–0.000
object	0.637	0.469	–0.002
libero-10	0.424	0.337	+0.001

6.3 Path B — cross-model recurrence

Activation-based recurrence (pooled data, 20 layer x suite cells)

Quantity	Value
`q_cross`	0.37 – 0.54
permutation floor `q_perm`	0.24 – 0.28
gap above floor	**+0.13 to +0.26 in all 20 cells**
confound `R^2` (base rate + inheritance)	0.001 – 0.113
discrimination (10th to 90th percentile)	0.27 – 0.69
chance-corrected retention, layers 0/8/16/24/31	0.587 / 0.619 / 0.635 / 0.585 / 0.473
same-model different-seed `q_seed`	0.640 / 0.657 / 0.629 / 0.612 / 0.531
`corr(q_cross, paper's P(general))`	–0.17 to +0.17 across all 20 cells

Causal-signature recurrence (action-position)

Quantity	Value
chance floor (random decoders through each model's own head)	0.226
top-decile (most general) minus rest — goal	–0.017
— spatial	–0.015
— object	–0.008
— libero-10	–0.011
discarded bin-permutation null	manufactured a fake gap of ≈ 0.33

The A x B join

Quantity	Value		
`corr(adjusted breadth, recurrence)`	–**0.127**		
measurement ceiling at breadth reliability 0.363	±0.602		
implied `\	r_true\	` lower bound	≥ 0.211
recurrence reliability needed for `\	r_true\	<= 0.30`	0.494

6.4 Behaviour — ablation and steering

`goal` coalition run (baseline 152/200 = 0.760 [0.696, 0.814])

Condition	Success	Damage	95% CI	b10/b01	p (exact)	MDE
firing	0.020	+0.740	[+0.679, +0.801]	148/0	< 0.001	0.169
general	0.710	+0.050	[–0.019, +0.119]	30/20	0.203	0.097
random	0.755	+0.005	[–0.065, +0.075]	26/25	1.000	0.098
specialist	0.785	–0.025	[–0.088, +0.038]	18/23	0.533	0.087

Design resolution: discordance rate 0.253; pooled MDE **9.7 points**; per-task MDE **22.6 points**; 79 episodes/task needed for a 5-point effect. Scope test (damage participation ratio) unresolved for every condition.

`goal` single-feature run (baseline 134/180 = 0.744)

Condition	Damage	95% CI	p (exact)
only_1134	–0.064	[–0.133, +0.005]	0.108
only_1140	–0.006	[–0.077, +0.064]	1.000
only_1167	–0.014	[–0.096, +0.067]	0.864
only_1235	–0.013	[–0.088, +0.063]	0.871
only_1628	+0.007	[–0.076, +0.090]	1.000
only_1999	+0.006	[–0.078, +0.090]	1.000

Pooled MDE 10.8 points, per-task 22.1. Damage participation ratio undefined for every condition — no task showed positive damage.

Steering (qualitative): steering toward a grasp-associated feature made the gripper close earlier in the episode.

6.5 B1 — action channels (`goal`)

Validation: recomputed argmax equals emitted token on **0.9919** of decisions.

Per-channel sufficiency

	dx	dy	dz	droll	dpitch	dyaw	gripper
features + bias	0.9403	0.9343	0.9048	0.9489	0.9204	0.9365	**0.9917**

	dx	dy	dz	droll	dpitch	dyaw	gripper
features alone	0.6066	0.8326	0.5931	0.9030	0.9227	0.7759	–**0.0461**
absolute causal-mass share	0.1353	0.1331	0.1377	0.1503	0.1453	0.1446	0.1537

Breadth versus channel

Quantity	Value
`corr(adjusted breadth, gripper concentration)`, partial	**+0.069**
per-channel partials	dx +0.081, dy +0.083, dz +0.195, droll +0.089, dpitch +0.099, dyaw +0.162, gripper +0.069
channel participation ratio	mean 3.21 of 7 (p10 1.33, p90 6.32)
`corr(task breadth, channel breadth)`	+0.238
general vs specialist profile difference	≤ 0.07 in any channel; P(g>s) 0.506 – 0.618
tie exposure of `base_rate`	**10.3% of values tied**

Necessity, given the feature fires

Group	dx (projection)	dx (coded)	gripper (projection)	gripper (coded)
general	0.0534	0.0569	0.0008	0.0007
random	0.0565	0.0564	0.0006	0.0007
specialist	0.0292	0.0342	0.0010	0.0011
firing	0.1284	0.1150	0.0197	0.0092

Projection versus coded ablation

Quantity	Value
projection flip rate, all decisions	0.0229 (over 176,654,016 feature-decisions)
projection flip rate, given the feature fires	0.0695 (over 16,586,298)
coded flip rate, all decisions	0.0060
coded flip rate, given the feature fires	0.0642
fraction of feature-decisions where the feature is active	9.4%
projection flips occurring when the feature did NOT fire	**71.5%**
coded flips occurring when the feature did not fire	–0.5% (i.e. zero — consistency check)

6.6 A4 — inventory-level recurrence

Geometry gate

Object	Effective rank	90% energy	99% energy
contrast-centred `W_U_act`	205.5	213	250
signature space occupied by goal's features	**50.8**	193	246
random 12-dim subspace explains	5.8% of an arbitrary direction		

Head comparison across the four models: `g` and `act_ids` identical; `W_U_act` differs by max 0.0266 (1.64x its own entry standard deviation). Impact on signatures: cosine 0.9967 – 0.9990 (worst single feature 0.964).

The m-sweep (target = goal)

	m=1	m=2	m=3	m=4	m=5	m=6	m=7	m=8
most-specialist decile	0.3065	0.3952	0.4507	0.4924	0.5268	0.5562	0.5822	0.6052
most-general decile	0.2828	0.3683	0.4248	0.4692	0.5051	0.5361	0.5637	0.5881
random floor	0.2464	0.3324	0.3924	0.4396	0.4788	0.5124	0.5420	0.5684
seed ceiling	0.4528	0.5278	0.5730	0.6070	0.6347	0.6584	0.6791	0.6974
chance-corrected retention	0.244	0.269	0.272	0.271	0.268	0.264	0.259	0.253

Four-suite slope comparison

Target	General slope	Specialist slope	Difference	Random-floor slope	General – floor
goal	+0.3053	+0.2987	+0.0066	+0.3220	–0.0167
spatial	+0.3040	+0.2947	+0.0093	+0.3213	–0.0173
object	+0.3049	+0.2985	+0.0064	+0.3218	–0.0169
libero-10	+0.3166	+0.3102	+0.0064	+0.3219	–0.0053

Anti-aligned fractions (share of features whose best absolute match points the opposite way): goal-vs-others 0.115 – 0.366; libero-10 highest against every partner (0.25 – 0.37).

Inventory clustering (target = goal)

k	Inventory match	Random floor	Seed ceiling	Retention	`corr(role breadth, match)`	`corr(role breadth, occupancy diff)`
8	0.270	0.203	0.547	+0.194	–0.810	–0.071
16	0.266	0.160	0.540	+0.279	–0.574	–0.517
32	0.227	0.170	0.397	+0.251	–0.537	–0.197

Stability across `k`: standard deviation 0.0192. Sliced Wasserstein: spatial 0.0208, object 0.0186, libero-10 0.0232, random 0.0180 – 0.0182, seed 0.0140.

Chapter 7 — What we claim

A result is not a claim. This chapter states exactly what the numbers in Chapter 6 do and do not support.

7.1 The central claim

> **Causal generality in a VLA's sparse-autoencoder features is real, concentrated, and replicable *within* a model — but it is a property of the task distribution, not a universal one. It does not predict whether a differently fine-tuned copy of the same base model contains the same feature, and the standard tools for measuring and testing it systematically misattribute it.**

7.2 Claims we make, with their support

C1. Firing statistics do not measure generality

The existing operational definition fails twice: the classifier's labels are not independent of its inputs (criterion contamination), and its central statistic is algebraically base firing rate under balanced task groups. A related metric, once base rate is controlled, predicts held-out firing **negatively** in all 20 cells tested — it is a memorisation signal.

Supported by §2.2, §6.1.

C2. Causal task-breadth is a valid, label-free generality measure

Breadth predicts a feature's causal importance on a task it was never measured on, beyond both causal magnitude and base firing rate, in **all four LIBERO suites**: +0.387 to +0.535, positive in 10/10 folds each, against a permutation floor of zero at 40 to 75 standard deviations. Attenuation correction puts the true relationship at `|r| >= 0.82`.

Supported by §4.8, §5.2, §6.2.

C3. Causal influence is sharply concentrated, and concentrated on the same features across tasks

An effective **102 of 2048** features carry the action in `goal` (106 spatial, 49 object, 9.6 libero-10). The top 50 features hold 44% to 73% of all causal mass. The **same top-50 recurs across tasks at 34 to 37 times chance**. Causal mass is more concentrated than firing rate in every suite, and column-shuffling destroys the effect — so this is about the same features being large across tasks, not about the shape of the distribution.

Supported by §5.3, §6.2.

C4. Causal generality is not localised to an action channel

The prediction that general features are gripper/phase controllers is **falsified**: the partial correlation with gripper concentration is +0.069, the lowest of the seven channels. There is, however, a second and largely independent breadth axis — features drive an effective 3.21 of 7 action dimensions, correlated with task breadth at only +0.238.

Supported by §5.9, §6.5.

C5. The gripper is a learned default, not a feature-driven decision

At a **frozen state**, the gripper's action margin is recovered almost entirely by the constant `mu + b_pre` bias (sufficiency 0.992) while the features contribute nothing (–0.046), where the other six channels sit at 0.59 to 0.92. Necessity agrees and is not tautological: given a feature fires, removing it changes a `dx` token 0.0534 of the time and a gripper token 0.0008 — about 65 times less. This identifies what the previously-unexplained "constant default-action bias of 0.405" is.

Supported by §5.9, §6.5. ***Scope****: this is the direct effect at a frozen state on one decode slot. It says nothing about trajectory effects, and is fully consistent with steering a grasp feature making the gripper close earlier, since features control the arm and the gripper follows the state.*

C6. Cross-model recurrence is real but weak, and general features recur least

Shared causal structure across independently fine-tuned models is above chance everywhere. But chance-corrected retention is only about **0.25** — changing the fine-tuning suite costs roughly three quarters of what changing an SAE random seed costs — and the most causally general features are, if anything, the least recurrent.

Supported by §4.12, §5.11, §5.12, §6.3, §6.6.

C7. That dissociation is not an artefact of one-to-one matching

Dictionary splitting was the strongest innocent explanation and it fails. Letting one model express another's role as a coalition of up to eight features improves matching **no more than a random dictionary** in any suite — every breadth decile rises with `m` more slowly than the random floor does, and chance-corrected retention is flat from `m = 1` to `m = 8`. The dissociation reproduces at the level of clustered role inventories (`corr(role breadth, match)` = –0.54 to –0.81), where measurement noise is far smaller.

Supported by §5.11, §5.12, §6.6.

C8. Three measurement results bound what this literature can currently claim

- Breadth rankings are only **0.27 to 0.47** reliable across disjoint task halves. The population axis is solid; individual feature rankings are weakly reproducible.
- **71.5%** of a standard projection-based ablation's effect lands on decisions where the feature never fired.
- At conventional episode budgets a coalition-ablation experiment cannot resolve damage below about **10 points**, so published null results in this design are uninterpretable without a power bound.

Supported by §5.4, §5.5, §5.10, §6.2, §6.4, §6.5.

7.3 Claims we explicitly do not make

Stating these is part of the result.

We do not claim	Why not
That general features are behaviourally load-bearing	The ablation is a **bounded null**: damage below 11.9 points against a design that resolves 9.7. It bounds the effect; it does not demonstrate one
That specialist features are inert	Same bound, same reason
That we know how many general features there are	A count needs a principled cut-point; the distribution was never tested for bimodality
That "independently trained models fail to converge"	The four models share a base checkpoint. They are divergent branches from a common ancestor, not independent draws. The finding is about **divergence under fine-tuning**, not failed universality
That the surviving ~25% recurrence is *rediscovered*	It might be inherited from the shared base. The inheritance control for causal signatures needs a base-model SAE, which does not exist
That the feature-level A x B null is established	At reliability 0.363 it is not defensible without a recurrence-reliability estimate (§5.6). The **role-level** version (C7) does not depend on that
That breadth predicts per-decision decisiveness	It does not (general ≈ random given firing) — but `adjusted_breadth` is residualised on magnitude by construction, so it never claimed to
Anything about other model families, other layers, or real robots	One model family, one layer, simulation only

7.4 Draft abstract

Sparse autoencoders are increasingly used to identify "general" features in vision-language-action models, but generality is operationalised through firing statistics and human labels. We show this definition is circular — the labels are screened by the same metrics the classifier regresses onto — and that under balanced task groups its central statistic reduces to base firing rate.

We redefine generality functionally and label-free as *causal influence that recurs*, separating two axes the standard definition conflates: breadth of causal influence across tasks within one model, and recurrence of causal signatures across separately fine-tuned models. Because OpenVLA's action readout is a dot product at

the final layer, per-feature contributions to the emitted action decompose exactly, giving both first-order attribution and exact counterfactuals over 446,096 decisions without a forward pass.

Causal task-breadth is real and replicates: it predicts held-out causal importance in all four LIBERO suites (partial rho = +0.39 to +0.53, 10/10 folds) against a permutation floor of zero. Influence is sharply concentrated — an effective 10 to 106 features of 2048 carry the action, with the same top-50 recurring across tasks at 34 to 37 times chance, and more concentrated than firing rate in every suite.

The second axis does not follow from the first. Fine-tuning a single base checkpoint on four different task suites yields dictionaries whose causal roles align only about 25% as well as two SAE seeds on the same model do, and the features that are most causally general within a model are the ones that align worst across them. This is not an artefact of dictionary splitting: allowing a model to express another's role as a coalition of up to eight features improves matching no more than a random dictionary does in any suite, and the dissociation reproduces at the level of clustered role inventories. Generality is therefore task-distribution-relative, not universal.

We also report three measurement results that bound what this literature can currently claim: breadth rankings are only 0.27 to 0.47 reliable across disjoint task halves; the standard projection-based ablation acts on decisions where the feature never fires 72% of the time; and at conventional episode budgets a coalition-ablation experiment cannot resolve damage below about ten points, making published null results uninterpretable without a power bound.

7.5 Why the failed experiments matter as much as the successful ones

Four experiments in this programme returned nothing, and three of those are load-bearing.

Experiment	Outcome	Why it counts
Firing-based generality metrics	Null	Closed an entire methodological route and forced the causal redefinition. Without it, the project would have measured base rate and called it generality
B1 — generality is channel-localised	Falsified	A sharp prediction, tested and refuted. It also produced C5 and the second breadth axis as by-products
A4 — splitting explains the Path B null	Failed to rescue	The strongest available alternative explanation, run to completion. Its failure is what upgrades C6 from a hedged observation into a defended finding
Coalition ablation	Bounded null	Only interpretable *because* the power bound was computed. Without it, the same data is unreportable

A programme that only reported its positive results would have published the firing metrics, would not have found the gripper result, and would have left the central dissociation permanently vulnerable to a one-line objection.

Chapter 8 — Limitations and known issues

Everything currently wrong, unresolved, or unverified. Nothing in this chapter has been fixed; it is recorded so that no reader is surprised by it later.

8.1 Scope limits

Limit	Consequence
One model family (OpenVLA-7B)	Nothing here establishes that the findings hold for diffusion or flow-matching policies. The attribution method itself depends on the readout being a dot product, which flow policies do not have
One layer (31, the last)	The exact decomposition is only available where the residual feeds the readout directly. Earlier layers need gradient or path-patching methods, a separate decision that was never taken
Simulation only	LIBERO is simulated. No claim transfers to a physical robot without testing
Four models sharing one base checkpoint	They are siblings, not independent draws. See §7.3
10 tasks per suite	Breadth is a participation ratio over 10 items, which is a coarse statistic. This is the direct cause of the low reliability in §8.2

8.2 The reliability ceiling

Breadth reliability is 0.27 to 0.47. This is the most pervasive limitation in the programme, because it bounds every claim about *individual* features:

- The features selected for ablation and steering were chosen from a ranking that reproduces only weakly across disjoint task halves. Part of any chosen coalition is noise, which dilutes any real effect the ablation might have found.
- The feature-level A x B correlation of −0.127 cannot be interpreted as a null without a recurrence reliability estimate (§5.6).
- Population-level claims (C2, C3) are unaffected: aggregating over 2048 features turns a weak per-feature signal into a decisive population result.

Recurrence reliability has never been measured. The cheapest estimate would recompute the cross-model match on disjoint halves of the probe frames and correlate the two rankings — no rollouts, no retraining.

8.3 The rank-tie defect (open, and it touches feature selection)

Rank correlations throughout the older pipeline compute ranks by sorting twice, which breaks ties by **array index** rather than averaging them (§3.3). On continuous data the two agree, which is why it went unnoticed.

It does not agree here. `base_rate` is a count over a fixed denominator, so rarely-firing features tie with each other. Measured on `goal`: **10.3% of feature values are tied.** Each tied block therefore receives an arbitrary ordering determined by feature index, inside `adjusted_breadth` — the ranking that selects every ablation and steering target.

A correct tie-averaging implementation exists and all Phase 2 code uses it. **The older pipeline is deliberately unchanged**, because fixing it moves published numbers, and that should be a deliberate decision rather than a side effect of an audit.

8.4 Issues found in the audit of Phase 2's own code

A line-by-line audit of the new analysis code was performed. Computations verified correct: the per-slot sufficiency statistic is algebraically identical to the original; the attribution gather reproduces the reference implementation exactly; both counterfactual semantics match brute-force recomputation through the real RMSNorm and unembedding; the retention arithmetic, the breakeven-reliability formula and the overlap chance term were all recomputed by hand and match the printed values; and the coded-flip consistency check (zero flips on non-firing decisions) passes on the real data.

The audit also found the following, none of which has been addressed.

A. Cluster-size confound in the role-level result — the most consequential

`corr(role breadth, match quality)` = –0.54 to –0.81 (§5.12) has **no control for cluster occupancy**. Larger clusters have more stable centroids, so they match better; they also have less noisy breadth means. Occupancy total-variation distances of 0.23 to 0.46 show the clusters are very uneven. If cluster size correlates with mean breadth, that correlation is partly or wholly a size effect. **This finding should be treated as unconfirmed until size is partialled out.**

B. The necessity table reports a pooled rate

The group flip rates in §6.5 are computed as (total flips) / (total opportunities) across a group's 100 features. That answers *"what is the probability that a randomly chosen active feature-decision flips?"* — an estimand dominated by the busiest features in the group. The mean of per-feature rates is a different number. The conclusion "general ≈ random > specialist" is stated under the pooled estimand and could move under the other.

C. The projection-versus-coded gap quotes the confounded column

The reported gap of 0.0168 uses all-decisions rates. The adjacent table now reports given-fires rates, where the two interventions differ by only about 0.005. The 0.0168 is not wrong, but it sits under a heading a reader will associate with the corrected table.

D. "Transitions" marks the whole timestep

The transition control flags all seven slots of a gripper-transition timestep. So "dx on transitions" means *dx at gripper-transition timesteps*, not *dx-channel transitions*. This is intended, but the labelling invites misreading.

E. The 0.81% argmax mismatch is unattributed

All 446,096 rows carry valid action tokens, so the 0.9919 agreement is genuine disagreement rather than filtered junk. The most plausible cause is float16 storage of the residuals flipping decisions that were near-ties. It does not corrupt the flip analysis, which uses the recomputed argmax as its own consistent baseline, but the cause is currently a guess.

F. The damage-participation-ratio null uses a clipped mean

Negative per-task damages are counted as zero before averaging, which slightly inflates the simulated damage. The direction is conservative for rejecting the null.

G. Latent, never triggered

The Hungarian implementation returns pairs in unsorted order when its input is transposed (more rows than columns). All current usage is square, so it has never fired.

8.5 Unexplained observations

Two results in the record have no explanation and should not be quoted as findings.

The sliced-Wasserstein anomaly. Random dictionaries sit *closer* to `goal`'s signature distribution (0.0180 to 0.0182) than any real fine-tuned model does (0.0186 to 0.0232). At the distribution level, the four models differ from each other more than from noise. We do not know why.

All seven channel partials are positive. The seven channel concentrations sum to 1 per feature, so their correlations with a common variable should partly cancel. Seven small positive values instead suggests a shared artefact. A shuffled-breadth control would settle it and has not been run.

8.6 Loose ends in the experimental record

Item	State
B1 channel analysis	`goal` only; the other three suites are three GPU jobs

Item	State
Second-seed SAEs	`goal` only for action-position data, so the "retains about a quarter" figure is `goal`-only even though the slope conclusion is four-suite
Inventory clustering	`goal` only as target
Inheritance control for causal signatures	Impossible without a base-model SAE, which has not been trained
Signature-sharpness control	Sharpness is now saved alongside the m-sweep but the decile contrast has not been residualised on it
Hungarian at scale for the original Path B	A standing commitment from the plan; the new implementation exists but has not been applied to the pooled-data results
`libero-10` anomalies	Extreme concentration (`n_eff` 9.6), highest anti-aligned fraction, and the only suite reversing the `m = 1` breadth ordering. Behaves differently on three axes and has not been investigated
Coalition baseline discrepancy	A working note records 78.9%; the data gives 76.0%. Unexplained

Chapter 9 — Glossary, file map, and reproduction

9.1 Glossary

Ablation. Removing something from a model to see what breaks. Here, removing a feature's direction from the residual stream. Two variants are distinguished: *projection* (remove everything along the direction) and *coded* (remove only the amount the SAE attributed to that feature).

Action. The seven numbers the policy outputs per decision: three translations, three rotations, one gripper command.

Adjusted breadth. Participation ratio over tasks, rank-residualised on causal magnitude and base firing rate. The ranking used for all feature selection.

Argmax. "The argument that maximises" — which item in a list is the largest. The model emits the token with the largest logit.

Attenuation. The fact that noisy measurements cannot correlate strongly. Corrected by dividing by `sqrt(r_xx * r_yy)`.

Base firing rate. The fraction of all decisions on which a feature is active. The confound that destroyed the firing-metric family.

Bin. One of the 256 buckets each action dimension is discretised into.

Bootstrap. Estimating uncertainty by resampling the data with replacement. *Two-level* means resampling at two nested units (tasks, then episodes within tasks).

Breadth. Effective number of tasks a feature's causal influence spreads over, measured by participation ratio.

Causal signature. The 256-number description of how a feature pushes every action bin: `S[j,t] = <w_j, g(*) u_t>`, contrast-centred. Depends only on the decoder and the readout, not on any input.

Chance-corrected retention. `(observed − floor) / (ceiling − floor)`. Expresses a similarity as a fraction of the most that was achievable.

Coalition. A set of features ablated or matched together.

Confound. A third variable that could explain a result without the interesting mechanism being true.

Contrast direction. `u_t` minus the average over all 256 action tokens. Removes the component that lifts every bin equally and therefore cannot affect the choice.

Criterion contamination. When the labels used to validate a measure are partly defined by the measure itself.

Discordant pair. In paired binary data, a pair where the two conditions disagreed. Only these carry information for McNemar's test.

Effective rank. Participation ratio of a matrix's squared singular values — how many directions it really spans.

Feature. A learned direction in the residual stream plus its activation. Indices are specific to one trained dictionary.

Flip. A counterfactual in which removing a feature changes which action bin the model emits.

Gini coefficient. Inequality summary from 0 (perfectly even) to 1 (all in one place).

LOTO / LOGO. Leave-one-task-out / leave-one-group-out. Train on all but one, test on the held-out one.

Logit. The unnormalised score the model assigns each token before choosing.

McNemar's test. The paired test for binary outcomes; asks whether the discordant pairs split more unevenly than a coin flip.

MDE. Minimum detectable effect — the smallest effect a design would catch 80% of the time. Anything smaller is below the experiment's resolution.

Null distribution. The range of values a statistic takes when the claimed effect is absent, generated by scrambling the data appropriately.

OMP. Orthogonal matching pursuit — greedily choose `m` items whose span best approximates a target direction.

Participation ratio (PR). `(sum x)^2 / (sum x^2)`. An effective count: 1 if all mass is in one place, `n` if spread evenly.

Partial correlation. Correlation remaining after removing what a control variable can explain.

Policy. A function from observation to action.

Reliability. How well a measurement agrees with itself when repeated. 0 = noise, 1 = perfectly repeatable.

Residual stream. The running 4096-dimensional vector each transformer layer adds to.

RMSNorm. Root-mean-square normalisation: divide by the vector's typical component size, then rescale by a learned gain.

Rollout. One run of the policy on one task from one starting state.

SAE. Sparse autoencoder — learns a dictionary of directions such that any input can be written as a sum of a few of them.

Slot. One of the seven decode positions. Channel identity is positional, so the slot determines which physical quantity a token refers to.

Spearman correlation. Pearson correlation computed on ranks; robust to outliers and to curved-but-increasing relationships.

Spearman–Brown. Corrects a split-half correlation for the fact that half a test is less reliable than the whole.

Sufficiency. The fraction of the action margin that a component of the SAE decomposition supplies, measured as a through-origin least-squares slope.

Superposition. Storing more concepts than there are dimensions by giving each a direction rather than an axis.

TopK. Sparsity by keeping only the `k` largest activations and zeroing the rest.

Unembedding. The matrix converting a residual vector into a score per vocabulary token.

Wilson interval. A confidence interval for a proportion that stays sensible near 0 and 1.

9.2 Code map

File	Role
`collect_action_activations.py`	A1 — collect action-position residuals and head constants
`mrvla/train_sae.py`	A2 — train the TopK SAE
`run_attribution.py`	A3 + A4 — the viability gate and causal task-breadth
`mrvla/attribution.py`	The attribution mathematics; `loto_partial_both` is the reported estimator
`identify_features.py`	Rank features by adjusted breadth; find causal exemplars
`capture_feature_frames.py`	Pull camera frames at those exemplars
`collect_shared_probe.py`, `run_recurrence.py`	Path B — activation-based recurrence
`run_causal_recurrence.py`	Path B — causal-signature recurrence and its random-decoder null
`join_pathA_pathB.py`	The A x B correlation
`run_ablation.py`, `mrvla/hooks.py`	Closed-loop ablation (projection semantics)
`run_steering.py`	Closed-loop steering
`split_half_breadth.py`	Breadth reliability
`mrvla/stats.py`	Wilson, McNemar (exact and corrected), paired CI, MDE, tie-aware ranks
`ablation_power.py`	Paired tests, damage intervals, power bound, two-level bootstrap
`permutation_null.py`	The two valid Path A floors, and a demonstration of the invalid one
`causal_concentration.py`	Effective feature count, Gini, cross-task overlap
`reliability_ceiling.py`	Attenuation correction and breakeven reliability
`action_space_geometry.py`	The A4 geometry gate and the shared-head check
`mrvla/readout.py`	Exact readout counterfactuals, both ablation semantics
`mrvla/channels.py`, `run_channel_attribution.py`	Slot-resolved attribution and per-feature necessity
`analyze_channels.py`	B1 statistics and controls
`mrvla/inventory.py`, `inventory_recurrence.py`	The m-sweep and its m-matched nulls
`mrvla/clustering.py`, `inventory_clusters.py`	Spherical k-means, Hungarian matching, sliced Wasserstein

Files in bold were added during Phase 2. All are covered by unit tests in `tests/`.

9.3 Reproducing the analysis

With the artifact root in `BASE`, the Phase 2 analyses run in this order.

1. The geometry gate (seconds, CPU). Decides whether the coalition sweep is meaningful.

```
python action_space_geometry.py \
--head goal=$BASE/ACT_ACTION/goal/head_constants.npz \
--head spatial=$BASE/ACT_ACTION/spatial/head_constants.npz \
--head object=$BASE/ACT_ACTION/object/head_constants.npz \
```

```
--head 10=$BASE/ACT_ACTION/10/head_constants.npz \
--sae goal=$BASE/ACT_ACTION_SAE/goal/sae --layer 31
```

2. The audit suite (minutes, CPU). Permutation floors, concentration, reliability, ablation power, attenuation.

```
BASE=$BASE bash run_diagnostics.sh
```

3. The channel pass (one GPU job) and its analysis (CPU).

```
sbatch run_channels.slurm goal
python analyze_channels.py --chan $BASE/CHANNELS/goal/layer_31_channels.npz \
--attr $BASE/ATTR/goal_k100/layer_31_attribution.npz
```

4. The inventory experiments (minutes, CPU; needs a second-seed SAE for the ceiling).

```
python inventory_recurrence.py --target goal \
--model goal=$BASE/ACT_ACTION_SAE/goal/sae [... one --model per suite ...] \
--head goal=$BASE/ACT_ACTION/goal/head_constants.npz [... one --head per suite ...] \
--attr $BASE/ATTR/goal_k100/layer_31_attribution.npz \
--seed-sae $BASE/ACT_ACTION_SAE/goal/sae_seed1 --layer 31 --m-max 8

python inventory_clusters.py --target goal [same --model/--head/--attr/--seed-sae] --k
8,16,32
```

Each model's decoder must go through **its own** head — the four unembeddings are not identical (§5.7).

9.4 Closing note

The through-line of this programme is that **a measurement is not trustworthy until you have tried to break it**. Four of the experiments recorded here returned nothing, and three of those four changed the direction of the work: the firing-metric null forced the causal redefinition, the channel result falsified a sharp prediction and produced two better findings as by-products, and the coalition-matching experiment failed to rescue a result and thereby made it defensible.

The single most reusable idea is probably the smallest one. Because the action readout is a dot product, the counterfactual "what would this model have done without this feature?" can be answered exactly, for every feature on every decision, without running the model at all. That turns a question which needed 200 noisy episodes into one answered by 176 million exact evaluations — and it is available in any model whose final operation is linear.